

PaperPass[免费版]查重报告

简明打印版

查重结果(相似度):

总体: 7%

本地库: 7% (本地库包含学术联合库、期刊库、学位库、会议库、共享联合库)

互联网: (免费版不检测互联网资源)

检测版本: 免费版(仅检测中文)

报告编号: 2PZ06A184F7E1775F

论文题目: 王帅毕业设计论文

论文作者: 佚名

论文字数: 29335

段落个数: 780

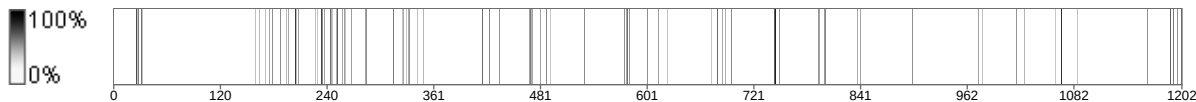
句子个数: 1202

提交时间: 2026-5-28 22:21:50

比对范围: 学术联合库、期刊库、硕博学位库、会议库、共享联合库

查询真伪: <https://www.paperpass.com/check>

句子相似度分布图:



本地库相似资源列表(学术联合库、期刊库、硕博学位库、会议库、共享联合库):

1. 相似度: 2.2% 来源: 学术联合库

东北电力大学

毕业设计说明书(论文)

作 者： 王帅 学 号： 2022303010533

学 院： 自动化工程学院 班 级： 自动卓越 221 班

专 业： ☒ 自动化

☐ 测控技术与仪器

☐ 机器人工程

所 在 系： ☒ 控制科学与工程 ☐ 仪器科学与技术

题 目： 用于电厂内巡检机器人的端到端无线视频传输

设计

指导者： 签字： :

评阅者： 1

2026 年 6 月 吉 林

摘 要

随着工业互联网、智能巡检和远程运维技术的不断发展，电厂等高风险、高复杂度场景对巡检作业的实时性、安全性和可追溯性提出了更高要求。传统人工巡检方式存在工作强度大、危险区域进入成本高、故障发现依赖人工经验等问题，而巡检机器人能够替代人员进入现场采集图像、视频和运行状态数据，为远程监控与智能诊断提供基础支撑。其中，稳定、低延迟的视频传输能力是巡检机器人系统能否有效应用的关键^[1]。

针对电厂巡检机器人在无线网络环境下实时视频回传存在的延迟高、弱网卡顿、设备状态难以统一管理等问题，本文设计并实现了一套端到端无线视频传输系统。系统采用前后端分离架构，由摄像头采集端、传输服务器、流媒体服务和 Web 监控客户端组成。摄像头采集端基于 Python 实现设备注册、心跳上报、摄像头采集和 WebRTC 推流^[2]；后端服务基于 FastAPI 实现用户认证、设备管理、设备在线状态监测和视频状态上报等功能^{[3][4]}；流媒体服务采用 MediaMTX 负责视频流接入、协议转换和 WebRTC 分发^[5]；前端采用 Vue3、Vite、TypeScript、Pinia 和 Element Plus 实现登录、设备管理、多分屏实时预览和基础系统设置等页面^{[6][7]}。

系统重点围绕实时视频预览和弱网自适应传输进行设计。通过 WHIP/WHEP 接入机制^{[8][9]}，摄像头端可将视频以 WebRTC 方式推送至 MediaMTX^{[2][5]}，浏览器端通过 WHEP 拉流完成低延迟播放^[9]。为提高弱网环境下的可用性，系统从采集参数、心跳检测、连接重试、播放端状态恢复等方面设计了自动降级与恢复机制，使系统在带宽下降或网络波动时优先保证视频连续性。

关键词：巡检机器人；实时视频传输；WebRTC；MediaMTX；FastAPI；Vue3；弱网自适应

ABSTRACT

With the rapid development of industrial Internet, intelligent inspection and remote operation technologies, power plants and other high-risk industrial scenarios have put forward higher requirements for real-time monitoring, operational safety and traceability. Traditional manual inspection methods suffer from high labor intensity, safety risks in hazardous areas and strong dependence on human experience. Inspection robots can enter industrial sites instead of workers and collect images, videos and status information, which provides important support for remote monitoring and intelligent diagnosis. Among these capabilities, stable and low-latency video transmission is one of the key factors that determine whether an inspection robot system can be effectively applied.

To solve the problems of high latency, video stuttering under weak networks and scattered device status management in wireless video transmission for power plant inspection robots, this paper designs and implements an end-to-end wireless video transmission system. The system adopts a front-end and back-end separated architecture, including a camera acquisition client, a transmission server, a media streaming service and a Web monitoring client. The camera client is implemented in Python and is responsible for device registration, heartbeat reporting, camera acquisition and WebRTC streaming. The back-end server is implemented with FastAPI and provides user authentication, device management, video status reporting and online status monitoring. MediaMTX is used as the media service for stream ingestion, protocol conversion and WebRTC distribution. The Web client is developed with Vue3, Vite, TypeScript, Pinia and Element Plus, providing login, device management, multi-screen live preview and basic system configuration pages.

The system focuses on real-time video preview and adaptive transmission under weak network conditions. By using WHIP/WHEP access mechanisms, the camera client can publish video streams to MediaMTX through WebRTC, while the browser client pulls streams through WHEP for low-latency playback. To improve availability under weak wireless networks, the system designs automatic degradation and recovery mechanisms from camera parameters, heartbeat detection, reconnection and playback state recovery, so that video continuity can be prioritized when bandwidth decreases or network jitter occurs.

Keywords: Inspection Robot; Real-time Video Transmission; WebRTC; MediaMTX; FastAPI; Vue3; Weak Network Adaptation

目录

摘 要	I
ABSTRACT	II
第 1 章 绪 论	1
1.1 课题背景与研究意义	1
1.2 国内外研究现状	2
1.3 研究内容与技术路线	3
1.3.1 研究内容	3
1.3.2 技术路线	3
1.4 论文结构安排	4
第 2 章 相关技术与理论基础	5
2.1 前后端分离架构	5
2.2 流媒体传输相关技术	5
2.2.1 WebRTC 协议	5
2.2.2 WHIP/WHEP 接入机制	5
2.3 系统开发技术	6
2.3.1 FastAPI 后端框架	6
2.3.2 Vue3+Vite 前端框架	6
2.3.3 MediaMTX 流媒体服务	6
第 3 章 系统需求分析	8
3.1 系统建设目标	8
3.2 业务场景分析	8
3.3 功能需求分析	9
3.3.1 用户登录与权限管理需求	9
3.3.2 设备注册与在线状态管理需求	9
3.3.3 实时视频预览需求	9
3.4 非功能需求分析	10
3.4.1 实时性需求	10
3.4.2 稳定性需求	10
3.4.3 可扩展性与可维护性需求	10
3.5 可行性分析	10
第 4 章 系统总体设计	12

4.1 总体架构设计	12
4.2 模块划分与通信关系	12
4.3 核心流程设计	13
4.3.1 设备注册流程	13
4.3.2 心跳与在线状态更新流程	14
4.3.3 实时推流与拉流流程	14
4.4 弱网自适应降级方案设计	15
4.5 系统安全性设计	16
4.5.1 用户认证与密码保护	16
4.5.2 设备接口认证	16
4.5.3 CORS 跨域访问控制	17
4.5.4 流媒体服务配置	17
4.5.5 安全性总结	17
4.6 数据库与接口设计	17
第 5 章 系统详细实现	19
5.1 后端模块实现	19
5.1.1 用户认证模块	19
5.1.2 设备管理与心跳模块	19
5.1.3 网络状态评估模块	20
5.2 摄像头采集端实现	20
5.2.1 摄像头采集与推流实现	20
5.2.2 弱网下动态参数调整实现	21
5.3 流媒体服务实现 (MediaMTX)	21
5.3.1 流接入与转发配置	21
5.3.2 弱网场景下的服务参数配合	22
5.4 前端模块实现	22
5.4.1 登录与设备列表页面	22
5.4.2 实时预览页面	23
5.4.3 性能监控组件	24
5.4.4 系统设置页面	25
6.1 测试环境与测试方法	28
6.2 功能测试	28
6.2.1 登录与设备管理功能测试	28
6.2.2 实时预览功能测试	29
6.2.3 自动降级与自动恢复功能测试	30

6.3 弱网专项测试	30
6.3.1 测试场景设计	30
6.3.2 带宽变化触发画质降级测试	31
6.3.4 连续波动网络下稳定性测试	33
6.4 性能观察	34
6.4.1 端到端时延观察	34
6.4.2 不同带宽下卡顿现象对比	35
6.4.3 CPU/内存占用观察	35
第7章 总 结	37
7.1 工作总结	37
7.2 核心成果与创新点	37
7.3 不足与后续展望	37
参考文献	39
附 录	40
致 谢	42

第 1 章 绪 论

1.1 课题背景与研究意义

电厂作为典型的大型工业场景，设备种类多、运行环境复杂、巡检点位分散，部分区域还存在高温、高压、强噪声、粉尘、狭小空间或电磁干扰等风险。长期以来，电厂巡检主要依赖人工定时巡查，巡检人员需要根据路线逐一检查设备状态，并通过肉眼观察、仪表读数和经验判断发现异常。该方式虽然灵活，但也存在巡检效率受人员经验影响大、危险区域进入成本高、异常留证不充分、远程协同能力弱等不足。

随着移动机器人、嵌入式计算、无线通信和视频处理技术的发展，巡检机器人逐渐应用于电力、化工、矿山、轨道交通等行业^[1]。机器人可搭载摄像头进入现场，完成图像采集和视频回传。对于远程监控人员而言，实时视频是理解现场情况最直接的数据形式，也是进行人工复核和异常判断的基础。因此，面向巡检机器人的视频传输系统不仅需要“能看见”，还需要满足低延迟、稳定性、弱网适应和设备可管理等要求^[10]。

在实际电厂现场，无线网络质量并不总是稳定。机器人移动过程中可能经过遮挡区域、拐角区域、信号衰减区域或干扰区域，带宽、时延、丢包率会出现波动。如果视频传输系统无法根据网络变化进行调整，就可能出现画面卡顿、播放中断、延迟累积或设备状态误判等问题，从而影响远程巡检的实时决策。因此，设计一套适用于巡检机器人场景的端到端无线视频传输系统具有一定的工程价值。

本文围绕“电厂巡检机器人端到端无线视频传输系统”展开研究与实现，系统综合使用 Python、FastAPI、Vue3、MediaMTX、WebRTC、WHIP/WHEP 等技术，构建从摄像头采集、流媒体传输、设备管理到浏览器播放的核心链路。

硬件平台方面，摄像头采集端部署在树莓派 5（RaspberryPi5）单板计算机上，搭载 USB 摄像头进行视频采集。树莓派 5 采用 64 位四核 Cortex-A76 处理器，主频 2.4GHz，配备 4GB 或 8GB LPDDR4X 内存，具备足够的计算能力完成视频采集、编码和 WebRTC 推流。无线通信采用 WiFi5（802.11ac）标准，支持 2.4GHz 和 5GHz 双频段，理论带宽可达 433Mbps，满足 720P 视频传输需求。后端服务器和 MediaMTX 部署在局域网内的服务器或云端，前端通过浏览器访问，无需额外硬件。

本课题的研究意义主要体现在理论意义和实际意义两个方面。

从理论意义上看，本文将 WebRTC 低延迟通信技术、流媒体服务器接入机制、前后端分离架构和设备心跳管理机制结合起来，查重 57%。形成了一套较完整的视频传输系统设计方法。系统不仅关注视频编码与播放，还将设备注册、在线检测、播放端连接恢复和视频状态上报等工程环节纳入统一设计，有助于理解实时视频系统从采集端到浏览器端的主要技术链路。

从实际意义上看，本系统可用于电厂巡检机器人远程监控场景。监控人员可以通过 Web 页面查看在线设备列表，选择巡检机器人或固定摄像头进行实时预览，在网络波动时系统可尽量保持画面连续性，从而提升巡检效率和安全性。



图 1-1 电厂巡检机器人视频回传应用场景示意图

1.2 国内外研究现状

实时视频传输一直是远程监控、视频会议、安防监控、无人机、机器人和工业互联网领域的重要研究内容。查重 57%。传统视频监控系统多采用 RTSP、RTMP、HLS 等协议。RTSP 适用于摄像头和流媒体服务器之间的实时流传输，具有实现成熟、设备兼容性强等优点；HLS 具有良好的浏览器兼容性和 CDN 分发能力，但通常存在数秒级甚至更高的延迟，不适合对实时性要求较高的巡检控制场景。。

近年来，WebRTC 因其低延迟、浏览器原生支持、支持拥塞控制和点对点媒体传输等特点，逐渐被应用于远程控制、云游戏、无人机视频回传、机器人遥操作和实时监控等领域^{[2][12]}。查重 44%。WebRTC 原本主要面向浏览器之间的音视频通信，随着媒体服务器和协议标准的发展，WebRTC 也开始用于“设备到服务器”和“服务器到浏览器”

的流媒体分发场景。WHIP 和 WHEP 分别用于 WebRTC 推流和拉流的 HTTP 接入过程^{[8][9]}, 降低了传统 WebRTC 信令设计的复杂度, 使流媒体服务器可以通过标准化接口接入 WebRTC 流。

国外在实时视频传输方面起步较早, 相关研究集中在低延迟编码、拥塞控制、端到端传输质量评估、多路视频分发和边缘计算等方向。部分工业机器人和无人系统已将 WebRTC 用于远程控制画面回传, 并结合自适应码率策略改善弱网场景下的用户体验。国内相关研究则更多结合具体应用场景展开, 例如智慧电厂、矿山巡检、变电站巡检、安防监控和无人机巡查等。在工程实践中, 常见方案包括基于 GB/T28181 的安防接入方案、基于 HLS/FLV 的网页播放方案, 以及基于 WebRTC 的低延迟播放方案。国内在巡检机器人应用、WebRTC 低延迟传输与工业场景视频监控等方面也有相关研究报道。

1.3 研究内容与技术路线

1.3.1 研究内容

查重 82%

本文的主要研究内容包括以下几个方面。

查重 43%

(1) 分析电厂巡检机器人实时视频传输的业务需求。围绕巡检机器人远程监控场景, 分析用户登录、设备注册、设备在线状态维护、实时预览和弱网恢复等功能需求, 并进一步归纳系统的实时性、稳定性、可扩展性和可维护性要求。

(2) 设计系统总体架构。系统采用前后端分离模式, 分为摄像头采集端、后端服务、流媒体服务和 Web 监控端四个部分。摄像头端负责采集与推流, 后端负责设备和用户管理, MediaMTX 负责视频流接入与转发, 前端负责展示设备列表和播放实时视频。

(3) 实现核心功能模块。后端基于 FastAPI 和 SQLAlchemy 实现认证、设备管理、心跳检测和视频状态上报, 采用 SQLite 轻量级数据库存储用户和设备信息; 摄像头端基于 Python、OpenCV、aiortc 等技术在树莓派 5 平台上实现视频采集和推流; 前端基于 Vue3、Vite、TypeScript、Pinia 和 Element Plus 实现交互界面; 流媒体服务基于 MediaMTX 完成 WebRTC 接入。

(4) 设计弱网自适应降级与恢复方案。系统在弱网条件下优先保证视频连续性, 通过基于心跳检测的分档降级机制, 在网络质量下降时降低分辨率、帧率和码率, 并在网络恢复后逐级回升。具体策略为: 连续心跳失败 1 次切换到 weak 档位, 连续失败 3 次切换到 poor 档位, 连续成功后恢复到 good 档位。系统通过重建推流连接应用新的编码参数, 同时配合心跳超时检测和播放端自动重连等方式提高传输稳定性。

(5) 开展系统测试与结果分析。通过功能测试、弱网专项测试和基础性能观察验证系统是否满足设计目标，并对端到端时延、播放稳定性等指标进行分析。

1.3.2 技术路线

本文采用“需求分析—总体设计—详细实现—系统测试—总结优化”的技术路线。首先根据电厂巡检机器人业务场景提取系统需求；然后确定前后端分离架构和基于WebRTC的低延迟传输方案；接着分别实现采集端、后端、流媒体服务和前端客户端；随后在局域网和弱网环境下进行功能测试与基础性能观察；最后总结系统成果与不足。

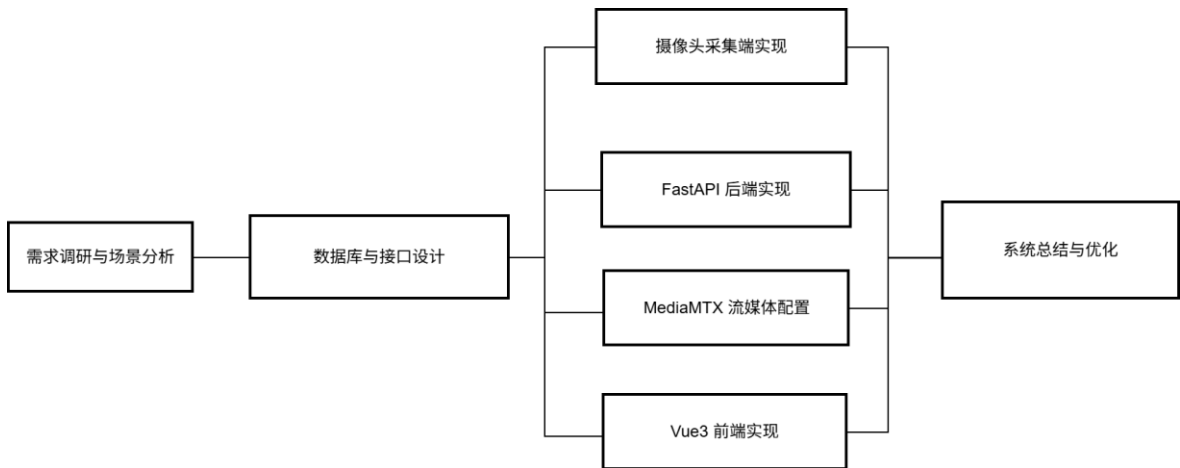


图 1-2 系统技术路线图

1.4 论文结构安排

本文共分为七章，各章内容安排如下。

第 1 章为绪论，主要介绍课题背景、研究意义、国内外研究现状、研究内容与技术路线以及论文整体结构。

第 2 章为相关技术与理论基础。

第 3 章为系统需求分析，从建设目标、业务场景、功能需求、非功能需求和可行性等方面分析系统设计依据。

第 4 章为系统总体设计，描述系统总体架构、模块划分、通信关系、核心流程、弱网自适应方案以及数据库和接口设计。

第 5 章为系统详细实现，分别介绍后端模块、摄像头采集端、MediaMTX 流媒体服务和前端模块的具体实现。

第 6 章为系统测试与结果分析，围绕功能测试、弱网专项测试和基础性能观察对系统效果进行验证。

第 7 章为总结，总结本文工作、核心成果和创新点，并分析不足与后续优化方向。

第 2 章 相关技术与理论基础

2.1 前后端分离架构

前后端分离架构是现代 Web 系统常用的软件架构模式。该模式将前端页面展示与后端业务处理分离，前端通过 HTTPAPI 与后端进行数据交互。前端主要负责界面渲染、用户交互、状态管理和路由跳转，后端主要负责业务逻辑、数据持久化、权限控制和接口服务。

在本系统中，前端采用 Vue3+Vite+TypeScript 开发，运行在浏览器中，负责登录页面、设备列表、实时监控和系统设置等主要用户界面。后端采用 FastAPI 开发，提供用户认证、设备注册、心跳上报、设备查询和视频状态上报等 RESTfulAPI。前后端通过 JSON 格式传输业务数据，视频流则由 MediaMTX 通过 WebRTC 通道独立传输。这样的设计使业务数据链路和视频媒体链路相对解耦，便于调试和维护。

前后端分离架构在本课题中的优势主要包括：

- (1) 前端和后端可以独立开发与部署，提高开发效率；
- (2) 接口边界清晰，有利于后续接入移动端、平板端或第三方平台；
- (3) 视频流媒体服务可独立运行，避免数据都经过业务后端而造成性能瓶颈；
- (4) 系统功能模块清晰，便于进行需求分析、设计说明和测试验证。

2.2 流媒体传输相关技术

2.2.1 WebRTC 协议

WebRTC 是一种支持浏览器和应用程序进行实时音视频通信的技术体系，具备低延迟、浏览器原生支持、拥塞控制、NAT 穿透和安全传输等特点^{[2][12]}。WebRTC 通过 RTCPeerConnection 建立媒体连接，并使用 SRTP 传输音视频数据^[12]。

对于巡检机器人远程监控场景，WebRTC 的主要优势在于低延迟。传统 HLS 播放通常存在较高分片延迟，而 WebRTC 可实现较低的端到端时延，更适合远程巡检人员及时观察设备状态。WebRTC 还具备网络拥塞控制能力，可以根据网络情况调整发送策略，从而提升弱网环境下的可用性。

2.2.2 WHIP/WHEP 接入机制

WHIP 主要用于将 WebRTC 媒体流通过 HTTP 接口推送到流媒体服务器^[8]。WHEP 主要用于客户端从服务器拉取 WebRTC 媒体流^[9]。两者都通过 HTTP 传输 SDPoffer/answer，简化了传统 WebRTC 应用中自定义信令服务器的开发^{[8][9]}。

在本系统中，摄像头采集端构造 WHIP 地址，格式为：

http://服务器地址:8889/设备 ID/whip 摄像头端创建 WebRTCoffer 后，将 SDP 发送至该 WHIP 地址，MediaMTX 返回 answer，双方完成 WebRTC 连接建立。前端播放端构造 WHEP 地址，格式为：

http://服务器地址:8889/设备 ID/whep 前端播放器通过 WHEP 地址拉取对应设备的视频流。由于推流路径和拉流路径均使用设备 ID 作为流名称，因此后端设备管理与流媒体播放能够通过统一的 device_id 关联起来。该机制降低了系统信令复杂度，也使流媒体链路更易部署和维护。



图 2-1 WHIP/WHEP 推拉流交互过程示意图

2.3 系统开发技术

2.3.1 FastAPI 后端框架

FastAPI 后端框架 FastAPI 是一个基于 Python 的现代 Web 框架，具有异步支持、类型提示友好、自动生成接口文档、开发效率高等特点。FastAPI 使用 Pydantic 进行数据校验，能够根据类型定义自动完成请求参数校验和响应模型生成；同时其基于 Starlette 和 ASGI，适合构建高性能异步 Web 服务。

2.3.2 Vue3+Vite 前端框架

Vue3 是渐进式 JavaScript 前端框架，具有响应式数据绑定、组件化开发和组合式 API 等特点^[6]。Vite 是新一代前端构建工具，利用浏览器原生 ESM module 实现快速开发启动和热更新，适合中小型前端应用快速开发^[6]。

2.3.3 MediaMTX 流媒体服务

MediaMTX 是一个轻量级、跨平台的流媒体服务器，支持 RTSP、RTMP、HLS、WebRTC、SRT 等多种协议^[5]。它可以接收来自摄像头或推流程序的视频流，并向不同客户端进行转发。

本系统使用 MediaMTX 作为独立流媒体服务，主要使用其 WebRTC 端口完成 WHIP 推流和 WHEP 拉流。MediaMTX 在系统中的作用包括：

- (1) 接收摄像头端的 WHIP 推流^[8]；
- (2) 将视频流转换为浏览器可播放的 WebRTC 流；
- (3) 按路径动态创建视频流，降低业务后端处理媒体数据的压力。

通过引入 MediaMTX，系统避免了在 FastAPI 后端中自行实现复杂的媒体转发逻辑，使后端可以专注于业务管理，而流媒体服务专注于音视频接入和分发，整体架构更加清晰。

第 3 章 系统需求分析

3.1 系统建设目标

本系统的建设目标是设计并实现一套适用于电厂巡检机器人场景的端到端无线视频传输系统。系统应能够完成摄像头视频采集、设备注册、设备在线状态维护、低延迟实时预览、弱网自动降级与恢复等核心功能，并提供清晰易用的 Web 管理界面。

具体目标包括：

- 1.实现摄像头采集端与服务器之间的设备注册和心跳通信，使平台能够实时掌握设备在线状态；
- 2.实现基于 WebRTC 的低延迟视频预览，满足巡检监控对实时性的要求；
- 3.以 WebRTC 为主要低延迟传输方案，完成采集端推流与浏览器端拉流；
- 4.实现 Web 端登录、设备管理、多分屏预览和系统配置等功能；
- 5.在弱网条件下通过参数调整、连接重试和状态恢复提升系统稳定性；
- 6.通过模块化结构降低后续维护成本。

3.2 业务场景分析

查重 51%

系统主要面向电厂巡检机器人远程视频监控场景。典型业务流程如下。

巡检机器人启动后，摄像头采集端首先向后端服务器注册设备，提交设备名称、设备类型、设备位置和 IP 地址等信息。后端为设备分配或返回 device_id，并生成对应的视频流地址。随后摄像头端启动心跳线程，定期向服务器上报告在线状态，同时启动摄像头采集和推流模块，将现场视频推送至 MediaMTX。

查重 63%

监控人员打开浏览器进入系统登录页面，输入账号密码完成认证。登录成功后，用户可进入设备管理页面查看在线设备、离线设备、设备位置、IP 地址、最后心跳时间等信息，也可进入实时监控页面选择在线设备进行播放。实时监控页面支持 1 分屏、4 分屏、9 分屏和 16 分屏布局，便于同时查看多台机器人或多个固定摄像头画面。

当机器人进入无线信号较弱区域时，视频流可能出现延迟增加、丢包、卡顿或连接断开。系统通过弱网降级策略降低视频参数，并通过播放端连接状态监听与自动重连机制恢复播放。后端设备监控服务会定期检查心跳时间，当设备超过一定时间未上报心跳时，将设备状态标记为离线，避免监控端误认为设备仍可用。

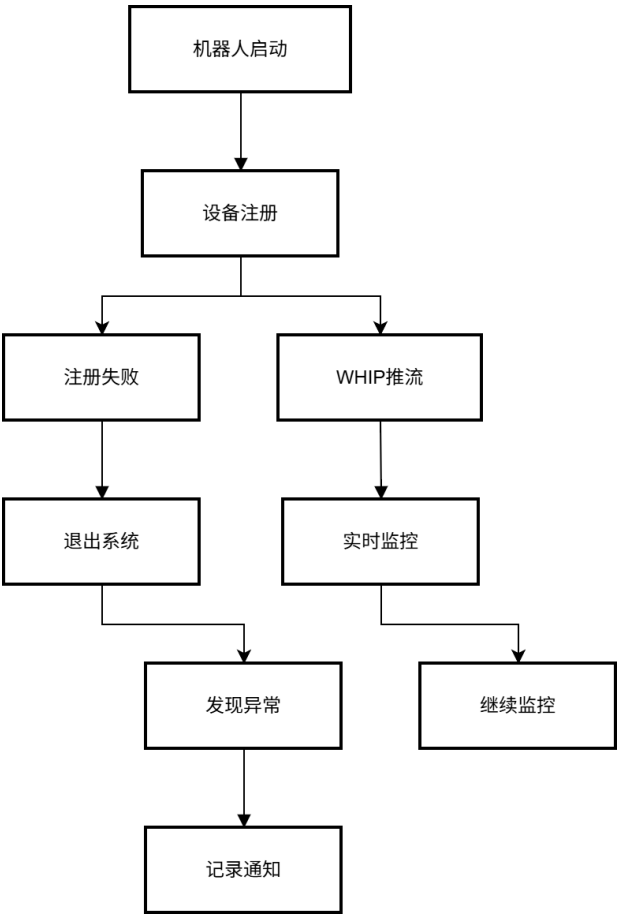


图 3-1 巡检机器人远程监控业务流程图

3.3 功能需求分析

3.3.1 用户登录与权限管理需求

查重 40% 系统需要提供用户登录功能，保证只有合法用户才能访问设备管理和实时监控页面。查重 49% 用户输入用户名和密码后，后端验证密码哈希，并签发访问令牌。前端保存令牌并在后续请求中携带，用于获取当前用户信息和访问受保护接口。

查重 72% 本系统当前内置默认管理员账号，用户名为 admin，密码为 admin123。后端用户模型包含 user_id、username、password_hash、role、created_at、updated_at 等字段。查重 40% 当前系统主要实现登录、获取当前用户、登出和修改密码等功能。

3.3.2 设备注册与在线状态管理需求

查重 48% 摄像头采集端启动后需要自动向服务器注册。查重 55% 注册信息包括设备名称、设备类型、设备位置和 IP 地址。服务器根据 IP 地址进行去重：如果该 IP 对应设备已经存在，则更新设备信息并将其状态置为 online；如果不存在，则创建新设备记录，并生成视频流地址。

查重 51%

设备注册成功后，采集端需要定期发送心跳请求。心跳接口接收设备 ID 和时间戳，并将设备的 last_heartbeat 更新为当前时间，同时将设备状态设置为 online。后端设备监控服务以固定间隔检查所有在线设备，如果发现某设备的最后心跳时间超过阈值，则将其状态置为 offline。用户也可以在设备管理页面查看设备列表、筛选设备状态、删除设备或跳转到实时监控页面查看视频。

3.3.3 实时视频预览需求

查重 42%

系统需要支持监控人员在 Web 页面中实时查看机器人视频画面。实时预览应具备以下能力：支持在线设备选择；支持 1/4/9/16 多分屏布局；支持播放加载状态和错误状态提示；支持全屏查看；支持关闭某一路视频；查重 44% 支持根据设备 ID 自动构造对应的 WHEP 播放地址。

为保证实时性，系统使用 WebRTC 作为浏览器端播放方式。前端播放器组件根据设备 ID 构造 WHEP 地址，并通过 WebRTC 播放器加载 MediaMTX 中对应的视频流。播放器组件需要在组件卸载时销毁连接，释放媒体轨道和浏览器资源，避免多路预览时资源泄漏。

3.4 非功能需求分析

3.4.1 实时性需求

查重 61%

实时性是本系统的核心非功能需求。电厂巡检人员需要根据视频画面及时判断设备异常，如果视频延迟过高，就会影响远程巡检的有效性。系统目标是在局域网或稳定无线网络下将端到端时延控制在 1 秒以内，在弱网情况下尽量避免延迟持续累积。

为满足实时性需求，系统采用 WebRTC 进行低延迟播放，编码端使用 zerolatency 调优参数和较小缓冲区^[8]，MediaMTX 直接负责视频转发，避免业务后端参与媒体数据转发。

3.4.2 稳定性需求

稳定性主要体现在设备长时间运行、网络波动和多路播放场景下系统仍能正常工作。采集端应在注册失败、推流失败或网络异常时输出日志，便于定位问题；后端应能稳定处理设备心跳和查询请求；前端应在播放失败时给出错误提示，并在连接中断时尝试恢复。

系统通过日志记录、心跳机制、设备监控服务、播放端重连和模块化部署提高稳定性。MediaMTX 作为成熟流媒体服务，也降低了自行实现媒体转发带来的不稳定风险。

3.4.3 可扩展性与可维护性需求

本系统采用前后端分离和模块化设计，后端按照认证、设备等模块划分接口，数据库模型清晰；前端按照视图、组件、状态仓库和 API 请求进行组织；流媒体服务独立部署。这样的结构便于后续维护，也便于在论文中进行系统设计说明。

3.5 可行性分析

从技术可行性看，系统采用的 FastAPI、Vue3、MediaMTX、WebRTC、FFmpeg、OpenCV、aiortc 等技术均较为成熟，具有较多实践案例和文档支持。系统核心功能可以在普通 PC、树莓派或局域网环境中完成验证，因此技术上可行。

从经济可行性看，系统主要使用开源框架和免费工具，不需要购买昂贵商业软件。硬件方面只需摄像头、普通服务器或开发主机、浏览器客户端即可完成基本测试，成本较低。

从操作可行性看，系统提供 Web 图形界面，用户可通过浏览器进行登录、设备查看和视频预览，操作方式简单直观。开发和部署文档中也给出了启动 MediaMTX、后端、前端和摄像头采集端的步骤，便于使用者上手。

从维护可行性看，系统各模块职责清晰，后端、前端、采集端和流媒体服务可以分别维护和替换。日志文件、接口文档和设备状态信息也为系统排查问题提供了基础。

第 4 章 系统总体设计

4.1 总体架构设计

本系统采用分层和模块化相结合的总体架构，由摄像头采集层、流媒体服务层、业务服务层和 Web 展示层组成。

摄像头采集层运行在巡检机器人或摄像头设备上，负责设备注册、心跳上报、摄像头采集、视频编码和推流。该层通过 HTTP 接口与后端通信，主流程通过 WHIP 将视频流推送到 MediaMTX。

流媒体服务层由 MediaMTX 实现，负责接收视频流、维护视频路径并进行 WebRTC 分发。该层是媒体数据传输的核心，不承担用户认证和设备业务逻辑。

业务服务层由 FastAPI 后端实现，负责用户认证、设备管理、设备在线状态监控、视频状态接收和接口文档生成。该层通过数据库持久化用户和设备等数据。

Web 展示层由 Vue3 前端实现，提供登录、主布局、设备管理、实时监控和设置等主要页面。前端通过 HTTPAPI 获取业务数据，通过 WHEP 地址从 MediaMTX 拉取视频流。

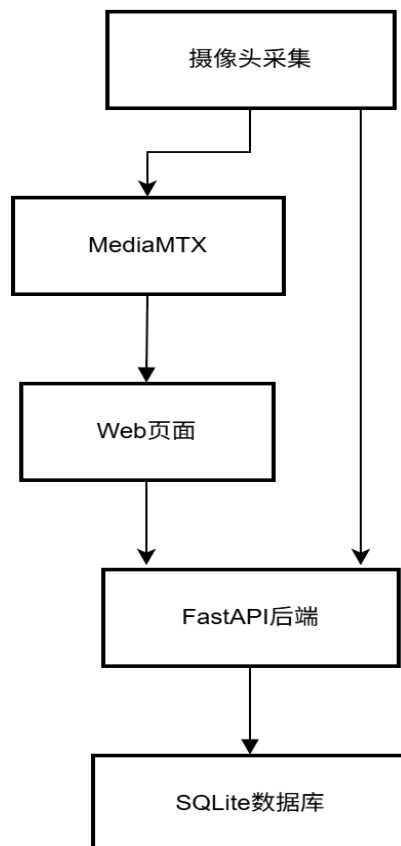


图 4-1 系统总体架构图

4.2 模块划分与通信关系

系统模块及通信关系如下。

摄像头采集端包括 DeviceManager、WHIPStreamer、CameraVideoTrack 等主要模块。DeviceManager 负责向后端注册设备、发送心跳和注销设备；WHIPStreamer 负责通过 aiortc 创建 WebRTC 推流连接；CameraVideoTrack 负责从摄像头获取帧并封装为 WebRTC 视频轨道。

后端服务包括 api_auth、api_devices、device_monitor、models 和 database 等主要模块。api_auth 负责用户登录和令牌校验；api_devices 负责设备注册、心跳、查询、下线、删除和视频状态上报；device_monitor 负责心跳超时检测；models 定义数据库表结构。

前端客户端包括 Login、Layout、DeviceManagement、LiveMonitor、Settings、VideoPlayer、stores 和 api 等主要模块。Login 页面负责登录；DeviceManagement 页面展示设备列表；LiveMonitor 页面提供多分屏视频预览；VideoPlayer 组件负责具体视频播放；Piniastore 负责用户、设备和本地配置状态管理；api 模块封装后端请求。

通信关系可概括为：采集端通过 HTTP 调用后端设备接口，通过 WHIP 推流至 MediaMTX；前端通过 HTTP 调用后端业务接口，通过 WHEP 从 MediaMTX 拉取视频流；后端与数据库交互维护业务数据；MediaMTX 根据配置进行媒体转发。

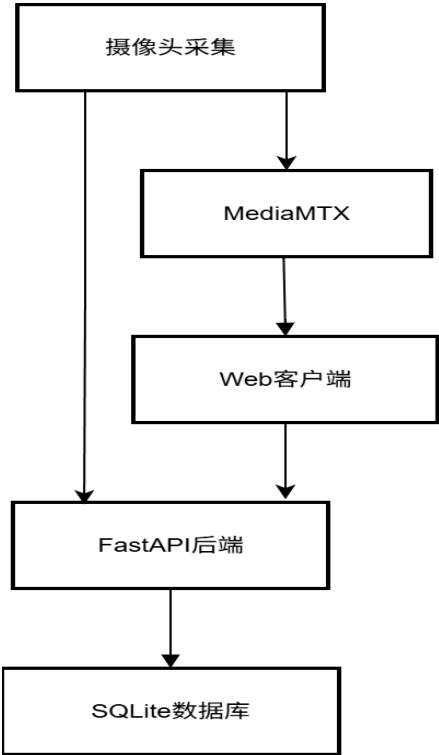


图 4-2 系统模块划分与通信关系图

4.3 核心流程设计

4.3.1 设备注册流程

设备注册流程是系统建立设备与视频流关联关系的第一步。摄像头采集端启动后，读取配置中的设备名称、设备类型和设备位置，并获取本机 IP 地址，然后向后端/api/devices/register 接口发送注册请求。

后端接收到注册请求后，首先根据 IP 地址查询设备表。如果已存在相同 IP 的设备，则认为该设备为重新上线，后端更新设备名称、类型、位置、在线状态和最后心跳时间，并检查 stream_url 是否有效；如果不存在，则创建新设备记录，生成唯一 device_id，并根据 MediaMTX 地址和设备 ID 生成流地址。注册成功后，后端返回 device_id、stream_url 和提示信息。

采集端收到 device_id 后，将其用于后续心跳上报和推流路径构造。例如 WHIP 推流路径为/设备 ID/whip，WHEP 拉流路径为/设备 ID/whep。通过该设计，设备业务信息与视频流路径形成统一映射。

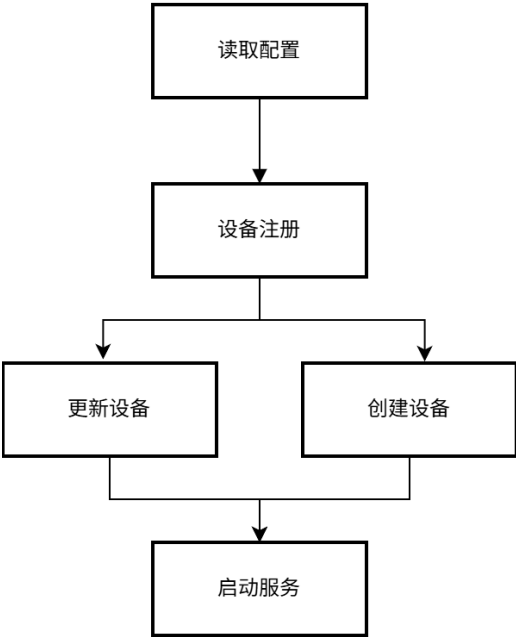


图 4-3 设备注册流程图

4.3.2 心跳与在线状态更新流程

设备注册成功后，采集端启动心跳线程，按照 10 秒的时间间隔向后端 /api/devices/{device_id}/heartbeat 接口发送心跳请求。请求中包含 timestamp 字段，用于表示设备当前上报时间。

后端接收到心跳后，根据 device_id 更新设备表中的 last_heartbeat 字段，并将状态设置为 online。如果设备 ID 不存在，则返回 404 错误。系统启动时，后端同时启动设备监控后台任务。该任务每隔 6 秒查询设备表中状态为 online 的设备，判断其

last_heartbeat 是否超过 30 秒超时阈值。如果设备长时间未发送心跳，则说明设备可能断网、关机或程序异常，系统将其状态更新为 offline。

通过心跳机制，前端设备列表可以实时反映设备在线状态，监控人员能够快速判断当前可用的视频源。

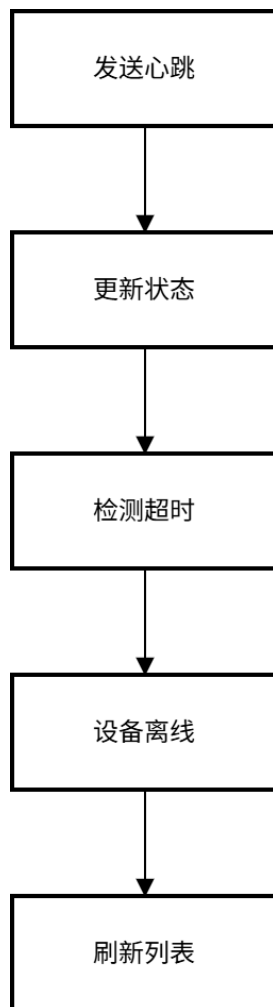


图 4-4 心跳检测与状态更新流程图

4.3.3 实时推流与拉流流程

系统以 WebRTCWHIP 推流作为低延迟主方案。在 WHIP 推流流程中，采集端根据 device_id 构造 WHIP 地址，然后创建 RTCPeerConnection，创建视频轨道并添加到连接中。随后采集端创建 WebRTCoffer，将本地 SDP 通过 HTTPPOST 发送到 MediaMTX 的 WHIP 端点。MediaMTX 返回 answerSDP 后，采集端设置远程描述，连接建立完成后开始持续推送视频帧。

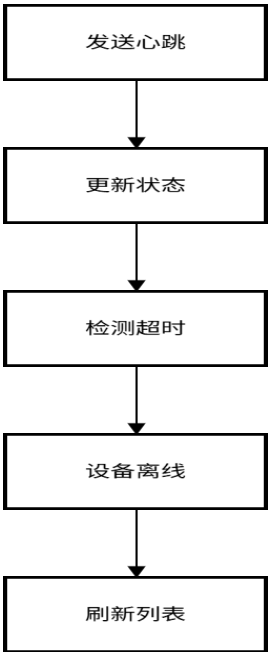


图 4-5 实时推流与拉流流程图

4.4 弱网自适应降级方案设计

弱网自适应是本系统的重要设计内容。巡检机器人处于移动状态时，无线网络可能因距离、遮挡、干扰和多设备竞争产生波动。系统弱网策略的目标不是在任何情况下都保持最高画质，而是在网络受限时优先保证视频连续性和可恢复性。

系统弱网自适应方案分为采集端、传输端、播放端和状态管理四个层面。

查重 44%
采集端层面，根据网络情况调整分辨率、帧率和码率。系统根据连续心跳失败次数判断网络质量：连续失败 1 次切换到 weak 档位，连续失败 3 次及以上切换到 poor 档位。当心跳连续成功达到阈值后，系统逐级恢复到 good 档位。档位切换通过重建 WHIP 推流连接实现，新连接使用对应档位的编码参数。

传输端层面，MediaMTX 负责视频流接入与转发，系统启用 WebRTC 端口用于低延迟拉流。对于弱网环境，WebRTC 本身具有一定拥塞控制能力，可在网络质量变化时调整媒体发送策略。MediaMTX 作为中间服务，可以减少多客户端直接连接采集端造成的压力。

播放端层面，前端播放器监听连接状态。如果 WebRTC 连接状态变为 failed 或 disconnected，系统延迟一小段时间后释放旧连接并重新发起拉流。这样可以在临时网络抖动后自动恢复画面，减少用户手动刷新页面的次数。

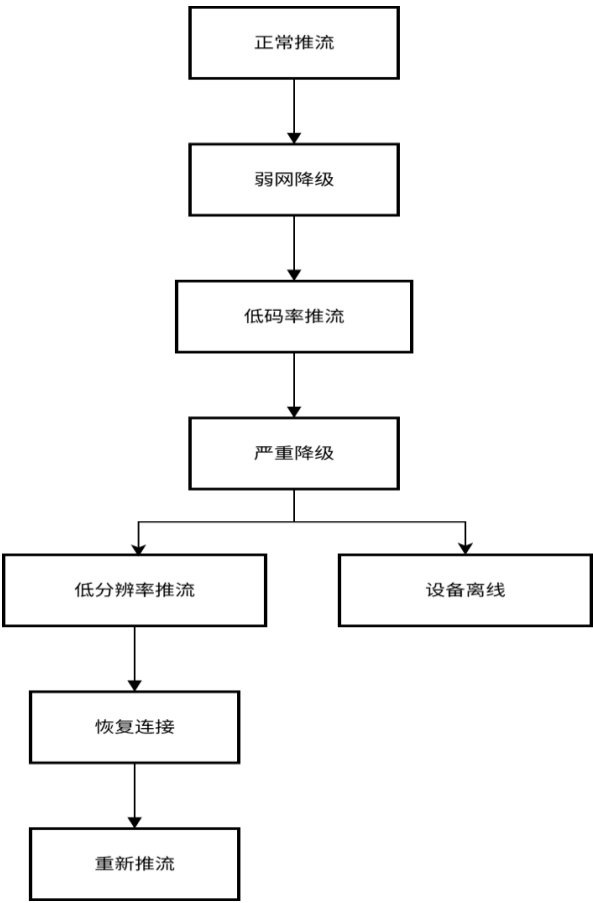


图 4-6 弱网自动降级与恢复策略示意图

4.5 系统安全性设计

系统在设计和实现过程中考虑了基础安全措施，主要包括用户认证、密码保护、设备接口认证、CORS 限制和流媒体配置安全等方面。

4.5.1 用户认证与密码保护

系统采用 JWT 进行用户认证。用户登录时，后端验证用户名和密码，验证成功后生成 JWT 令牌返回给前端。前端将令牌保存在本地存储中，后续请求通过 HTTP 请求头携带令牌进行身份验证。JWT 令牌设置有效期为 24 小时，过期后需要重新登录。

密码存储方面，系统使用 bcrypt 算法对用户密码进行哈希处理后存储到数据库中，避免明文存储密码。bcrypt 是一种自适应哈希函数，具有计算成本可调、抗彩虹表攻击等特点，适合用于密码存储。

系统启动时会检测是否使用默认密钥，如果检测到默认 JWT 密钥或默认管理员密码，会在日志中输出警告信息，提示生产环境必须修改默认配置。

4.5.2 设备接口认证

为防止未授权设备注册和伪造心跳，系统在设备相关接口（设备注册、设备心跳、视频状态上报）中增加了设备 API 令牌认证机制。采集端在调用这些接口时，需要在 HTTP 请求头中携带 X-Device-Token 字段，后端验证令牌是否正确。只有持有正确令牌的采集端才能注册设备和上报状态。

设备 API 令牌通过环境变量 DEVICE_API_TOKEN 配置，采集端和后端使用相同的令牌值。生产环境部署时应修改默认令牌，并妥善保管令牌值。

4.5.3 CORS 跨域访问控制

查重 50%

系统后端配置了 CORS（跨域资源共享）中间件，用于控制哪些前端地址可以访问后端 API。开发环境可以配置为允许所有来源，方便局域网内多设备访问。系统启动时会检测 CORS 配置，如果配置为允许所有来源，会在日志中输出警告信息。

4.5.4 流媒体服务配置

MediaMTX 配置文件中增加了安全说明注释，说明系统部署在局域网内，通过网络隔离保证安全，设备注册和心跳通过后端 API 进行认证。

生产环境建议启用 MediaMTX 的 publish 和 read 认证功能，对推流和拉流进行访问控制。查重 59% 同时建议启用 HTTPS 加密传输，防止视频流在网络传输过程中被窃听。

4.5.5 安全性总结

系统实现的安全措施包括：JWT 用户认证、bcrypt 密码哈希、设备 API 令牌认证、CORS 白名单限制、关闭未使用功能、默认配置安全提示等。这些措施能够满足局域网内部署的基本安全需求。

查重 40%

生产环境部署时，还应考虑以下安全增强措施：启用 HTTPS 加密传输、配置 MediaMTX 流媒体鉴权、定期更新密钥和令牌、限制数据库访问权限、配置防火墙规则仅允许必要端口访问、定期审计系统日志等。

4.6 数据库与接口设计

系统采用 SQLite 轻量级数据库存储用户和设备信息，数据库主要包含用户表和设备表，围绕登录认证、设备管理、心跳检测和视频状态上报展开。

用户表 users 用于保存登录用户信息，主要字段保存 bcrypt 哈希后的密码，避免明文存储。这样前端设备列表能够直接展示当前弱网等级、重连次数和视频流状态，不需要额外查询其他表。

系统核心接口包括：

POST/api/auth/login： 用户登录；

GET/api/auth/me： 获取当前用户信息；

POST/api/auth/logout： 用户登出；

POST/api/auth/change-password： 修改密码；

POST/api/devices/register： 设备注册；

POST/api/devices/{device_id}/heartbeat： 设备心跳；

GET/api/devices： 获取设备列表；

GET/api/devices/{device_id}： 获取设备详情；

POST/api/devices/{device_id}/offline： 设备下线；

DELETE/api/devices/{device_id}： 删除设备；

POST/api/devices/{device_id}/stream-status： 上报视频流状态、弱网等级和当前推流参数。

表 4-1 系统核心 API 接口表

接口路径	请求方法	功能	说明
/api/auth/login	POST	用户登录	校验用户名和密码， 返回访问令牌
/api/auth/me	GET	当前用户信息	根据令牌返回用户基础信息
/api/auth/logout	POST	用户登出	清理前端登录状态， 后端记录登出行为
/api/auth/change	POST	password	修改密码
/api/devices/register	POST	设备注册	采集端启动后注册设备并获取 device_id
/api/devices/{device_id}/heartbeat	POST	设备心跳	更新设备最后心跳时间和网络基础指标
/api/devices/{device_id}/stream	POST	status	视频状态上报
/api/devices	GET	设备列表	按状态或设备类型查询设备
/api/devices/{device_id}	GET	设备详情	查询单个设备信息
/api/devices/{device_id}/offline	POST	设备下线	采集端退出时将设备标记为离线
/api/devices/{device_id}	DELETE	删除设备	删除不再使用的设备记录

第 5 章 系统详细实现

5.1 后端模块实现

5.1.1 用户认证模块

用户认证模块位于后端 `api_auth.py` 中，主要负责登录、令牌生成、当前用户识别、登出和修改密码等功能。^{查重 56%} 登录流程中，后端首先根据用户名查询用户表，如果用户不存在或密码校验失败，则返回 401 错误。^{查重 50%} 密码校验使用 `bcrypt`，将用户输入密码编码后与数据库中的 `password_hash` 比对。^{查重 66%} 验证成功后，后端调用 `create_access_token` 方法生成 JWT 访问令牌，令牌载荷中包含用户名和过期时间。^{查重 55%} 前端登录成功后保存 `access_token`，并在后续请求中携带令牌。^{查重 50%} 当前用户获取功能通过 `get_current_user` 实现。该方法从请求头中解析 token，使用系统配置的算法解码 JWT，然后根据用户名查询用户表。如果令牌无效、过期或用户不存在，则返回认证失败。修改密码接口先验证旧密码，再生成新密码哈希并写入数据库。

用户认证模块保证了系统基本访问安全。



图 5-1 登录页面截图

5.1.2 设备管理与心跳模块

设备管理模块位于 `api_devices.py` 中，核心功能包括设备注册、心跳更新、设备列表查询、设备详情查询、设备下线和设备删除。

设备列表接口支持按 `status` 和 `device_type` 过滤，查询结果按创建时间倒序排列。前端设备管理页面通过该接口展示设备名称、类型、位置、IP 地址、状态、分辨率、帧率和最后心跳时间等信息。删除设备接口用于移除不再使用的设备记录，下线接口用于在采集端退出时将设备状态置为 `offline`。

设备在线状态监控由 `device_monitor.py` 实现。该模块在后端启动时作为后台任务运行，每隔 6 秒检查一次在线设备的心跳时间。若当前时间与 `last_heartbeat` 的差值超过 30 秒，则将设备状态更新为 `offline`。该机制可有效处理采集端异常退出或网络中断导致设备状态无法及时更新的问题。

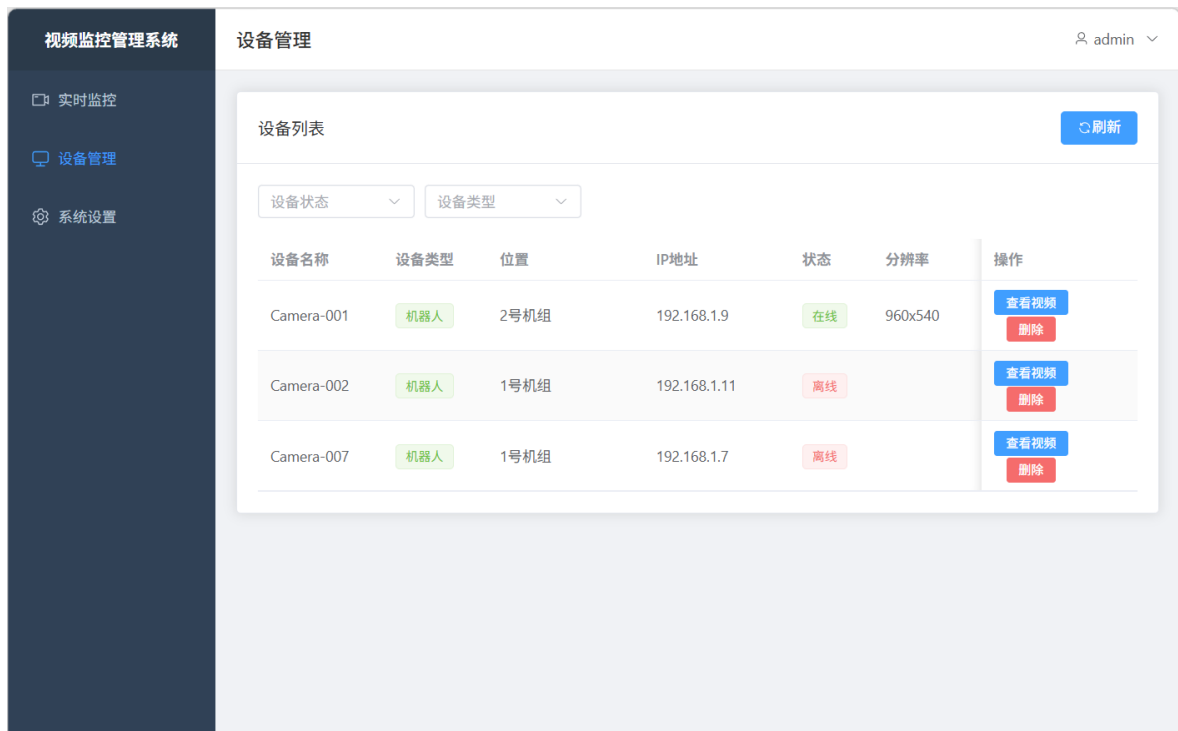


图 5-2 设备管理页面截图

5.1.3 网络状态评估模块

网络状态评估模块主要服务于弱网降级与恢复策略。在当前系统实现中，网络状态评估以设备心跳、推流连接状态、播放端连接状态和推流参数为主要依据。

后端通过心跳时间判断设备是否仍与平台保持连接。若心跳连续超时，则说明设备可能处于离线或严重弱网状态，系统将设备状态置为 `offline`。前端通过设备状态和播放器事件判断视频流是否可用，若播放器触发 `no-media`、连接失败或断开，则向用户展示错误提示并尝试重连。

采集端可根据网络情况调整推流参数。系统将网络状态划分为 `good`、`weak` 和 `poor` 三个等级。心跳连续失败时进入降级档位，连续恢复后回到正常档位。WHIP 推流模块采用 WebRTC 连接方式，可利用 WebRTC 自身拥塞控制机制改善网络波动下的传输表现。

由于需要在复杂度和可验证性之间取得平衡，本文将网络状态评估分为三个等级：正常、弱网和严重弱网。正常状态下使用 720P 和较高帧率；弱网状态下降低码率和帧率；严重弱网状态下降低分辨率并触发重连或离线提示。

5.2 摄像头采集端实现

5.2.1 摄像头采集与推流实现

摄像头采集端由 Python 实现，主程序位于 camera-client/main.py。程序启动后依次完成日志配置、设备管理器初始化、设备注册、心跳启动、推流器初始化和主循环运行。设备管理器封装设备注册、心跳和注销逻辑。 register 方法会获取本机 IP 地址，并向后端注册接口发送设备信息。注册成功后保存 device_id。start_heartbeat 方法创建后台线程，按固定间隔调用心跳接口。程序退出时，cleanup 方法停止推流、停止心跳并调用 unregister 将设备置为离线。

WHIP 推流模块位于 whip_streamer.py。WHIPStreamer 启动时创建 offer，将本地 SDP 发送到 MediaMTX 的 WHIP 端点，收到 answer 后完成远程描述设置，从而建立 WebRTC 推流连接。

```
(venv) yuxiang@yingraspberrypi:~/Desktop/video/camera-client $ python main.py
2026-05-23 18:15:37 | INFO | __main__:setup - 摄像头采集端启动 mode=WHIP
2026-05-23 18:15:37 | INFO | device_manager:register - 注册设备 name=Camera-001 type=robot location=2号机组 ip=192.168.1.9
2026-05-23 18:15:38 | INFO | device_manager:register - 设备注册成功 device=940d3c37-ebc6-4f0f-ad1a-7d55ad728103
2026-05-23 18:15:38 | INFO | device_manager:start_heartbeat - 心跳已启动，间隔：10秒
2026-05-23 18:15:38 | INFO | __main__:start_streamer - 启动推流 profile=good resolution=1280x720 fps=25 bitrate=2000kbps
2026-05-23 18:15:38 | INFO | whip_streamer:start - 启动WHIP推流 url=http://192.168.1.5:8889/940d3c37-ebc6-4f0f-ad1a-7d55ad728103/whip resolution=1280x720 fps=25 bitrate=2000kbps
[0:37:16.617594729] [3257] INFO Camera camera_manager.cpp:330 libcamera v0.5.2+99-bfd68f78
[0:37:16.627989059] [3265] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:50)
[0:37:16.630520813] [3265] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/share/Libcamera/ipa/rpi/pisp/ov5647.json
[0:37:16.637404768] [3265] INFO Camera camera_manager.cpp:220 Adding camera '/base/axi/pcie@120000/rp1/i2c@88000/ov5647@36' for pipeline handler rpi/pisp
[0:37:16.637427064] [3265] INFO RPI pisp.cpp:1179 Registered camera /base/axi/pcie@120000/rp1/i2c@88000/ov5647@36 to CF E device /dev/media0 and ISP device /dev/media3 using PiSP variant BCM2712_C0
[0:37:16.637475045] [3265] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:50)
[0:37:16.638991605] [3265] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/share/Libcamera/ipa/rpi/pisp/imx290.json
[0:37:16.640989198] [3265] INFO Camera camera_manager.cpp:220 Adding camera '/base/axi/pcie@120000/rp1/i2c@80000/imx290@1a' for pipeline handler rpi/pisp
[0:37:16.641012920] [3265] INFO RPI pisp.cpp:1179 Registered camera /base/axi/pcie@120000/rp1/i2c@80000/imx290@1a to CF E device /dev/media1 and ISP device /dev/media4 using PiSP variant BCM2712_C0
[0:37:16.643794691] [3257] INFO Camera camera.cpp:1215 configuring streams: (0) 1280x720-RGB888/Rec709/Rec709/None/Full (1) 1920x1080-GBRG_PISP_COMP1/RAW
[0:37:16.643940967] [3265] INFO RPI pisp.cpp:1483 Sensor: /base/axi/pcie@120000/rp1/i2c@88000/ov5647@36 - Selected sensor format: 1920x1080-SGBRG10_1X10/RAW - Selected CFE format: 1920x1080-PC1g/RAW
2026-05-23 18:15:38 | INFO | device_manager:heartbeat_loop - 心跳正常 device=940d3c37-ebc6-4f0f-ad1a-7d55ad728103 rtt=52.66ms net=good
2026-05-23 18:15:39 | INFO | whip_streamer:start - WHIP推流已启动
2026-05-23 18:15:39 | WARNING | __main__:report_stream_status - [档位变化] init -> good | 分辨率: 1280x720 | 帧率: 25fps | 码率: 2000kbps | RTT: 52.66ms | 丢包率: 0.0% | 重连次数: 0 | 流状态: active
2026-05-23 18:15:39 | INFO | __main__:setup - 初始化完成
2026-05-23 18:15:39 | INFO | __main__:run - WHIP推流运行中...
2026-05-23 18:15:39 | INFO | whip_streamer:on_connectionstatechange - WHIP连接状态 state=connecting
2026-05-23 18:15:39 | INFO | __main__:report_stream_status - 采集状态 stream=connecting net=good rtt=52.66ms loss=0.0% reconnects=0 profile=1280x720@25fps/2000kbps
2026-05-23 18:15:39 | INFO | whip_streamer:on_connectionstatechange - WHIP连接状态 state=connected
2026-05-23 18:15:39 | INFO | __main__:report_stream_status - 采集状态 stream=active net=good rtt=52.66ms loss=0.0% reconnects=0 profile=1280x720@25fps/2000kbps
2026-05-23 18:17:19 | INFO | device_manager:heartbeat_loop - 心跳正常 device=940d3c37-ebc6-4f0f-ad1a-7d55ad728103 rtt=38.15ms net=good
2026-05-23 18:18:37 | WARNING | whip_streamer:on_connectionstatechange - WHIP连接状态 state=closed
2026-05-23 18:18:37 | INFO | __main__:report_stream_status - 采集状态 stream=reconnecting net=good rtt=3105.75ms loss=0.0% reconnects=0 profile=1280x720@25fps/2000kbps
2026-05-23 18:18:40 | INFO | __main__:report_stream_status - 采集状态 stream=reconnecting net=good rtt=3105.75ms loss=0.0% reconnects=1 profile=1280x720@25fps/2000kbps
```

图 5-3 摄像头采集端运行日志截图

5.2.2 弱网下动态参数调整实现

弱网下动态参数调整的核心思想是根据网络质量降低视频数据量，避免在带宽不足时仍强行传输高码率视频导致卡顿和延迟累积。查重 41%系统可调整的关键参数包括分辨率、帧率、码率、GOP 长度和编码缓冲区。

正常网络下，采集端使用 1280×720 分辨率、25fps 帧率和 2000kbps 码率。当心跳出现失败或推流连接进入异常状态时，系统会重建 WHIP 推流连接并切换到较低档位。弱网档位使用 960×540、20fps、1200kbps；严重弱网档位使用 640×360、15fps、600kbps。网络连续恢复后，系统再切回正常档位。

在实现层面，本系统已经将分辨率、帧率和码率作为配置项集中管理，并在推流模块中作为初始化参数传入，为动态调整提供了基础。播放端则通过连接失败后的重连机制配合恢复，当弱网恢复或参数调整后，用户可继续查看实时画面。

表 5-1 弱网降级参数配置表

网络等级	触发条件	分辨率	帧率	码率	处理策略
good	心跳连续成功	1280x720	25fps	2000kbps	实时预览
weak	出现心跳失败	960x540	20fps	1200kbps	降低码率和分辨
poor	连续心跳失败	640x360	15fps	600kbps	使用低码率档位
recovery	心跳恢复成功	逐级恢复	逐级恢复	逐级恢复	重新建立 WHIP

5.3 流媒体服务实现（MediaMTX）

5.3.1 流接入与转发配置

当摄像头端使用 WHIP 推流时，MediaMTX 根据路径接收 WebRTC 推流；当浏览器端使用 WHEP 拉流时，MediaMTX 输出 WebRTC 流，该设计使设备 ID 与视频流路径保持一致。

5.3.2 弱网场景下的服务参数配合

在弱网场景下，MediaMTX 的主要作用是稳定接入与转发，避免多客户端直接连接采集端。系统将流媒体服务与业务服务分离，使视频转发压力集中在 MediaMTX 上，后端 FastAPI 不直接处理媒体数据，减少业务接口受视频流量影响的风险。

```
yuxiang@yingraspberrypi:~$ cd ~/mediamtx
./mediamtx
2026/05/23 17:39:00 INF MediaMTX v1.18.1, linux, arm64
2026/05/23 17:39:00 INF configuration loaded from /home/yuxiang/mediamtx/mediamtx.yml
2026/05/23 17:39:00 INF [RTSP] listener opened on :8554 (TCP/RTSP), :9000 (UDP/RTP), :9001 (UDP/RTCP)
2026/05/23 17:39:00 INF [RTMP] listener opened on :1935 (TCP/RTMP)
2026/05/23 17:39:00 INF [HLS] listener opened on :8888 (TCP/HTTP)
2026/05/23 17:39:00 INF [WebRTC] listener opened on :8889 (TCP/HTTP), :8189 (UDP/ICE)
2026/05/23 17:39:00 INF [SRT] listener opened on :8890 (UDP)
```

图 5-4 MediaMTX 启动与流接入日志截图

5.4 前端模块实现

5.4.1 登录与设备列表页面

前端登录页面负责采集用户名和密码，并调用后端接口获取访问令牌。登录成功后，前端将令牌保存到状态管理或本地存储中，并跳转到系统主页面。后续 API 请求通过统一请求封装携带令牌，提高代码复用性。

设备管理页面使用 ElementPlus 的表格组件展示设备列表，包含设备名称、设备类型、位置、IP 地址、状态、分辨率、帧率、码率、网络等级、视频流状态、最后心跳和操作按钮等列。页面顶部提供状态和设备类型筛选条件，用户可按在线、离线、机器人、固定摄像头等条件筛选设备。用户点击“查看视频”后，系统选择该设备并跳转到实时监控页面；点击删除按钮时，页面弹出确认框，确认后调用删除接口。设备列表数据由 Pinia 管理，页面挂载时获取后端设备列表。该设计避免多个页面重复维护设备数据，有利于状态共享和代码维护。

设备列表 刷新

设备状态 设备类型

设备名称	设备类型	位置	IP地址	状态	分辨率	帧率	码率	网络	视频流	最后心跳	操作
Camera-001	机器人	2号机组	192.168.1.9	在线	960x540	20fps	1200kbps	降级	重连中	2026/5/23 10:25:39	查看视频 删除
Camera-002	机器人	1号机组	192.168.1.11	离线	-	-	-	未知	未传输	2026/5/7 07:46:39	查看视频 删除
Camera-007	机器人	1号机组	192.168.1.7	离线	-	-	-	未知	未传输	2026/5/7 07:48:05	查看视频 删除

图 5-5 前端设备列表页面截图

5.4.2 实时预览页面

实时预览是系统 Web 端的核心页面。页面顶部提供分屏布局切换按钮，包括 1 分屏、4 分屏、9 分屏和 16 分屏。中间区域为视频网格，每个网格单元可选择一个在线设备进行播放。未选择设备时，单元格显示“点击选择设备”；点击后弹出设备选择对话框，仅展示在线设备。

当用户选择某台设备后，页面在对应网格中创建 VideoPlayer 组件。VideoPlayer 根据 device_id 构造 WHEP 地址，格式为：

http://服务器地址:8889/device_id/whep

然后使用 WebRTC 播放器加载该地址。播放过程中，组件显示加载状态；若发生错误，则显示错误遮罩；用户可点击全屏按钮查看大画面，也可点击关闭按钮释放该路视频。

页面还实现了设备选择的本地保存功能，将已选择设备 ID 保存到 localStorage。用户刷新页面后，系统可根据已保存设备 ID 恢复布局中的设备选择，提高使用体验。页面还支持根据配置进行设备列表自动刷新，以便及时反映设备在线状态变化。

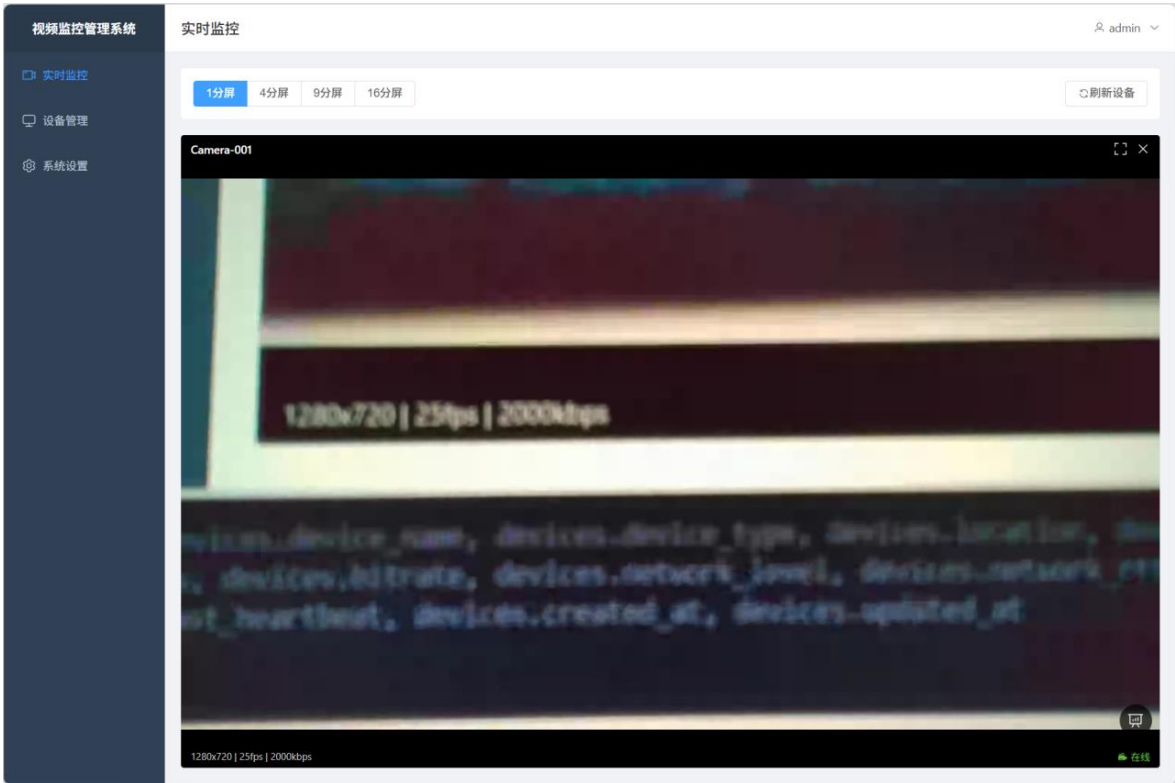


图 5-6 实时监控多分屏页面截图-1 分屏

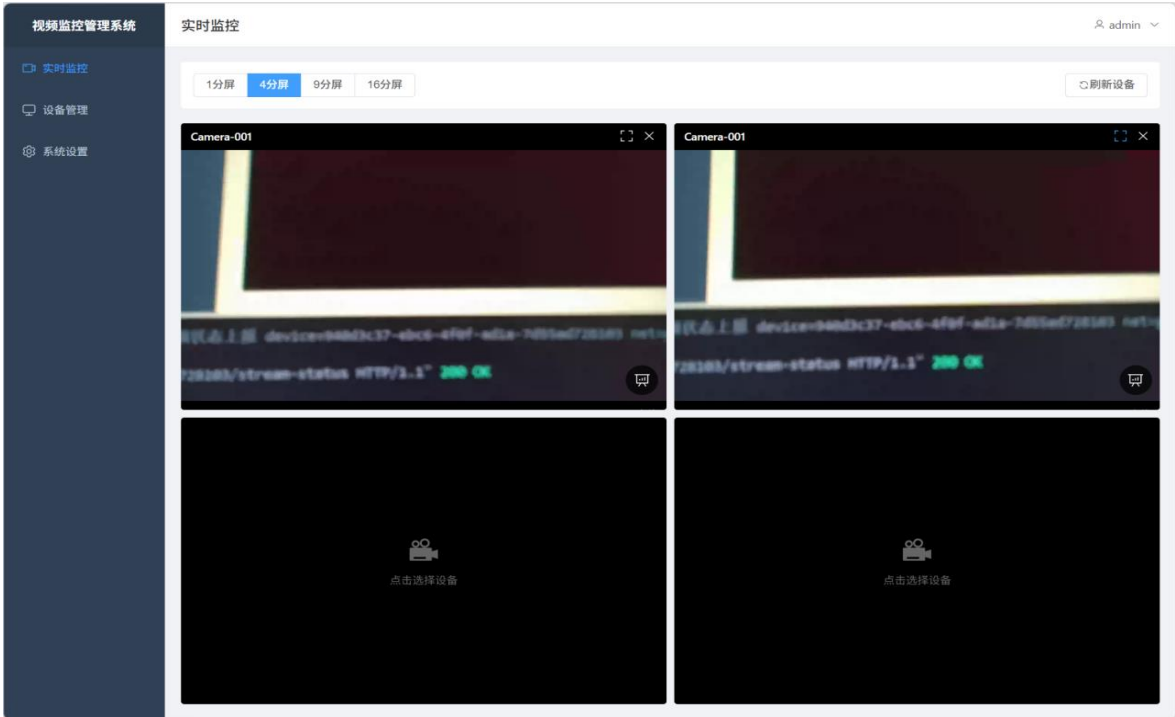


图 5-7 实时监控多分屏页面截图-4 分屏

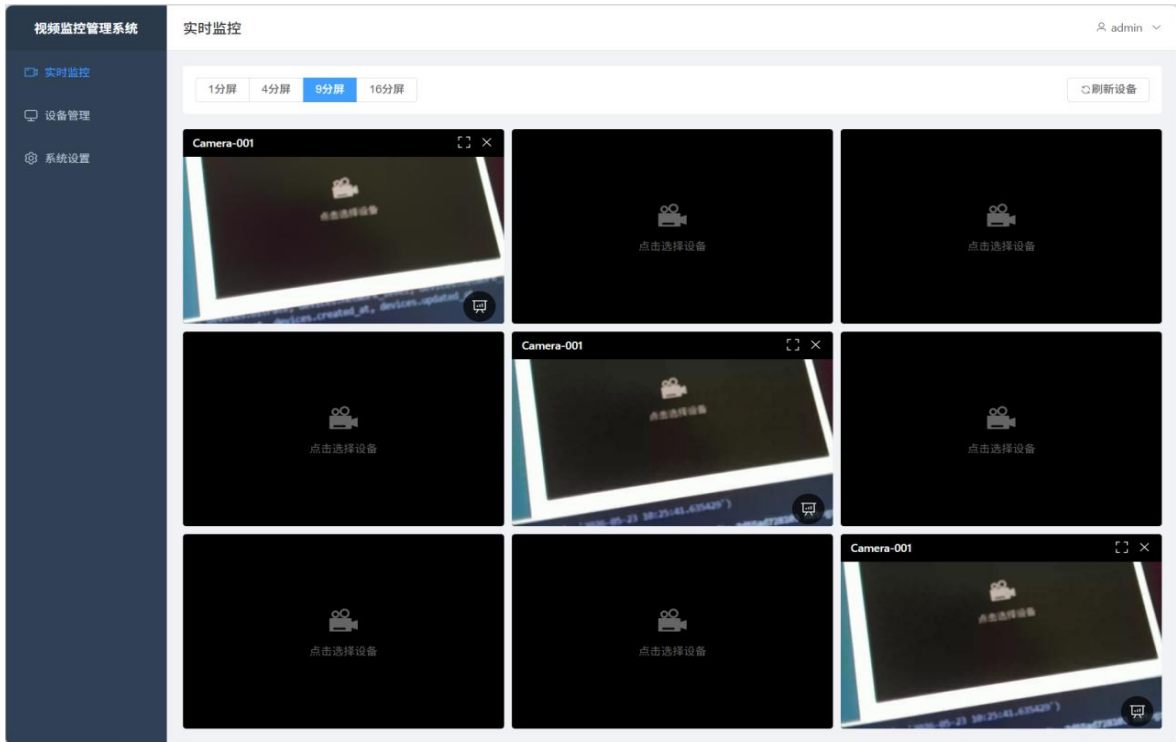


图 5-8 实时监控多分屏页面截图-9 分屏

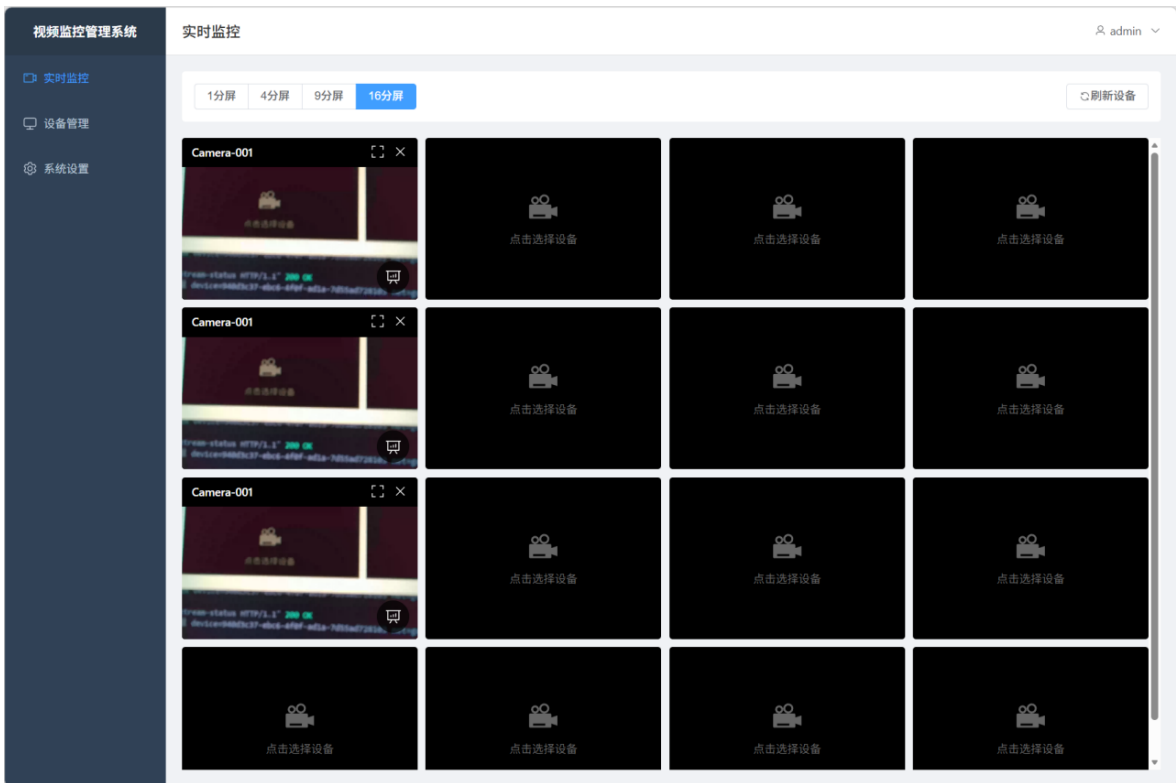


图 5-9 实时监控多分屏页面截图-16 分屏

5.4.3 性能监控组件

为了获取真实的视频传输性能数据，系统在前端播放器组件中集成了实时性能监控功能。性能监控组件在视频播放界面右下角提供性能监控按钮。用户点击按钮后，系统显示性能监控面板，展示以下实时指标：

(1) 基础视频参数：分辨率、帧率、码率。分辨率和帧率从统计报告中获取，码率通过计算两次采样之间的字节差值得出。

(2) 网络质量指标：丢包率、RTT、抖动。丢包率根据丢失包数和接收包数计算得出，RTT 从统计报告中获取，抖动反映网络传输的稳定性。

(3) 统计数据：丢包数、接收包数、网络档位。网络档位显示当前采集端的降级状态（良好/一般/较差），并根据档位使用不同颜色标识。

性能监控组件每秒调用一次 `getStats` 方法，从 `RTCPeerConnection` 对象中提取统计数据。关键代码逻辑如下：

```
``javascript
async function getVideoStats(peerConnection) {
  const stats = await peerConnection.getStats()
  stats.forEach(report => {
    if (report.type === 'inbound-rtp' && report.kind === 'video') {
      fps = report.framesPerSecond
      packetsLost = report.packetsLost
      packetsReceived = report.packetsReceived
      bytesReceived = report.bytesReceived
      resolution = `${report.frameWidth}x${report.frameHeight}`
    }
    if (report.type === 'candidate-pair' && report.state === 'succeeded') {
      rtt = report.currentRoundTripTime * 1000
    }
  })
}
``
```

该组件为系统测试提供了真实数据来源，避免测试数据的主观估算。在弱网测试中，可以通过性能监控面板直接观察丢包率、RTT 和档位变化，并截图作为测试证据。

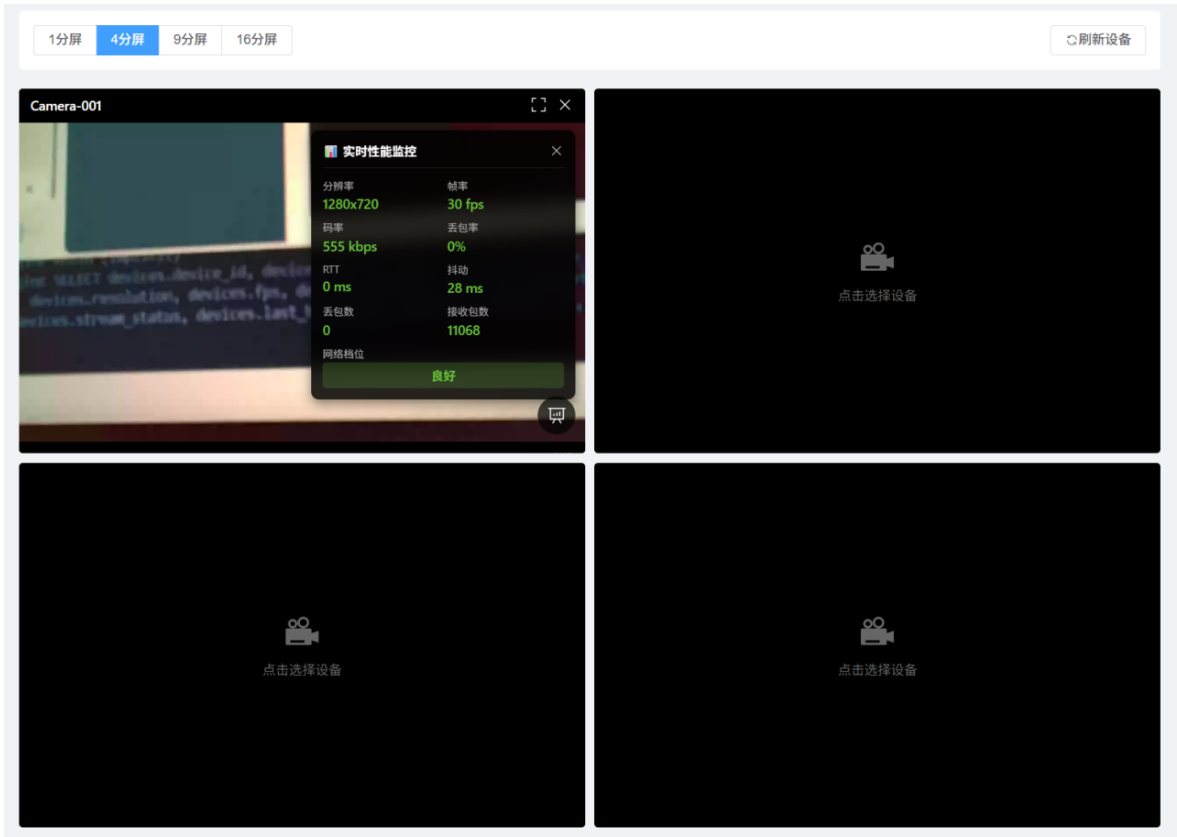


图 5-10 前端性能监控面板截图

5.4.4 系统设置页面

系统设置页面提供基本设置、用户信息、修改密码和关于系统等功能模块。

基本设置模块允许用户配置设备列表自动刷新间隔、是否显示性能监控面板等选项。
修改密码模块允许用户修改自己的登录密码。用户需要输入当前密码、新密码和确认新密码。
系统会验证当前密码是否正确，检查新密码和确认密码是否一致。

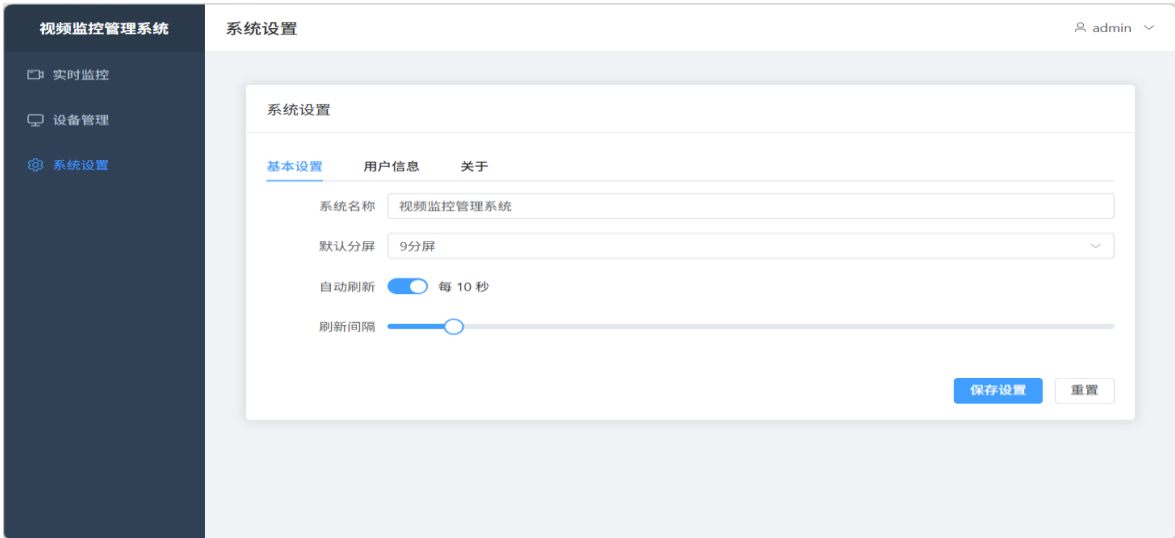


图 5-11 系统设置-基本设置截图

第 6 章 系统测试与结果分析

6.1 测试环境与测试方法

查重 55%
为验证系统功能和性能，本文在局域网环境下搭建测试环境。测试硬件包括一台开发主机、一台摄像头设备或树莓派摄像头、普通 USB 摄像头以及无线网络环境。软件环境包括 Python3.9 版本、Node.js18 版本、FFmpeg、MediaMTX、Chrome 浏览器、FastAPI、Vue3 和相关依赖库。

测试启动顺序为：首先启动 MediaMTX 流媒体服务；然后启动 FastAPI 后端服务；接着启动 Vue3 前端开发服务；最后启动摄像头采集端。启动完成后，通过浏览器访问前端地址，登录系统并进行设备管理和视频预览测试。

测试方法包括功能测试、弱网专项测试和基础性能观察。功能测试采用黑盒测试方式，从用户角度验证登录、设备注册、设备列表、实时预览和自动恢复等功能是否正常。弱网专项测试通过限制带宽、增加网络波动或降低采集参数模拟实际无线网络变化，观察视频播放连续性和恢复情况。性能观察主要关注端到端时延和播放稳定性，不作为严格实验室基准测试。

表 6-1 系统测试环境配置表

类别	配置项	内容
硬件环境	开发主机	Windows 桌面环境
硬件环境	摄像头设备	树莓派 5（4GB 内存）
网络环境	正常网络	局域网环境
网络环境	弱网模拟	参数模拟网络波动
后端环境	Python	Python3.9
前端环境	Node.js	Node.js18
流媒体环境	MediaMTX	WebRTC 端口 8889
浏览器环境	浏览器	Chrome

6.2 功能测试

6.2.1 登录与设备管理功能测试

登录测试中，输入正确账号 admin 和密码 admin123，系统应返回访问令牌并进入主界面；输入错误密码时，系统应提示用户名或密码错误，并拒绝访问。测试结果表明，系统能够正确完成登录校验和错误处理。

设备注册测试中，启动摄像头采集端后，采集端向后端注册设备，后端返回 device_id 和 stream_url。刷新设备管理页面后，可看到新增设备状态为在线，设备名称、类型、位置、IP 地址和最后心跳时间显示正常。停止采集端后，设备心跳停止，后端监控任务在超时后将设备状态更新为离线。

设备管理测试中，用户可按设备状态和设备类型进行筛选；点击“查看视频”可跳转到实时监控页面；点击删除按钮并确认后，设备记录从列表中移除。测试结果表明设备管理功能满足设计要求。

表 6-2 登录与设备管理功能测试用例表

测试项	操作步骤	预期结果	测试结果
正确登录	输入 admin/admin123	登录成功进入监控	通过
错误登录	输入错误密码	密码错误	通过
设备注册	启动采集端	状态为在线	通过
设备心跳	采集端持续运行	最后心跳时间持续	通过
设备离线	停止采集端或断开网络	状态变为离线	通过
设备筛选	按在线状态	符合条件的设备	通过
删除设备	点击删除并确认	设备从列表中移除	通过

6.2.2 实时预览功能测试

实时预览测试主要验证多分屏布局、设备选择、视频播放、全屏和关闭等功能。测试时，用户进入实时监控页面，选择 1 分屏、4 分屏、9 分屏或 16 分屏布局，点击空白单元格打开设备选择对话框，选择在线设备后，页面开始加载视频。

在视频流正常的情况下，播放器能够从 WHEP 地址拉取视频并显示实时画面。点击全屏按钮后，浏览器进入全屏播放状态；点击关闭按钮后，当前网格释放视频资源并恢复为空白单元格。刷新页面后，系统可根据本地保存的设备 ID 恢复已选择设备。

测试结果表明，实时预览页面能够完成多路视频监控的基本交互，适合巡检监控人员查看机器人现场画面。



图 6-1 实时视频预览测试截图

6.2.3 自动降级与自动恢复功能测试

自动降级与恢复测试主要验证弱网情况下系统是否能够尽量保持可用。测试过程中，通过降低采集端码率、帧率和分辨率模拟降级过程，通过临时断开网络或停止推流模拟异常。

当网络短时波动导致 WebRTC 连接失败或断开时，播放端可触发重连逻辑，释放原连接并重新发起拉流。若采集端恢复推流，前端视频可重新显示。若设备长期无法发送心跳，后端将设备状态置为离线，设备列表同步显示离线状态。

测试结果说明，系统能够在一定程度上处理网络波动和设备异常，具备基本的自动恢复能力。

表 6-3 自动降级与恢复测试结果表

测试场景	触发条件	系统行为	结果
轻度弱网	出现一次心跳失败	推流参数降低	通过
严重弱网	连续多次心跳失败	使用低码率档位	通过
连接中断	WebRTC 连接 failed	播放端触发重连	通过
网络恢复	心跳连续成功	网络恢复为 good	通过
设备离线	心跳超过阈值未恢复	设备标记为 offline	通过

6.3 弱网专项测试

6.3.1 测试场景设计

弱网测试设置正常网络、轻度弱网、中度弱网和严重弱网四类场景。正常网络下带宽充足，系统使用 720P、25fps 参数；轻度弱网下降低码率但保持 720P；中度弱网下降低帧率和码率；严重弱网下降低分辨率并观察重连和离线状态。

测试指标包括视频是否可播放、端到端时延、是否出现明显卡顿、播放是否中断、恢复时间和设备状态是否正确。测试时使用 NetLimiter 等网络限速工具限制带宽，模拟弱网环境。同时通过前端性能监控面板实时观察丢包率、RTT、帧率等指标，通过采集端日志记录档位变化过程。

测试方法：

- (1) 正常网络：不限制带宽，观察 good 档位运行状态
- (2) 轻度弱网：使用 NetLimiter 限制带宽到 1.5Mbps，触发 weak 档位
- (3) 严重弱网：继续限制带宽到 800kbps，触发 poor 档位
- (4) 网络恢复：取消带宽限制，观察档位恢复过程

每个场景下，同时记录：

- 前端性能监控面板的实时数据（分辨率、帧率、码率、丢包率、RTT）
- 采集端日志中的档位变化记录
- 视频播放的主观体验（流畅度、清晰度）

表 6-4 弱网测试场景设计表

场景	网络状态	推流参数	观察重点
正常网络	局域网稳定	1280x720、25fps、 2000kbps	画面清晰度和播放延 迟
轻度弱网	偶发心跳失败	960x540、20fps、 1200kbps	是否自动降级，播放 是否连续
严重弱网	连续心跳失败	640x360、15fps、 600kbps	是否进入低码率档 位，是否减少中断
网络恢复	心跳连续成功	恢复至正常档位	是否自动恢复画质

6.3.2 带宽变化触发画质降级测试

带宽变化测试中，系统在正常带宽下先以 720P 参数运行，观察视频画面和时延；随后逐步降低可用带宽，并将采集端码率从较高值降低到中低值，同时适当降低帧

率。测试结果表明，当码率降低后，画面清晰度有所下降，但播放连续性明显优于高码率强行传输的情况。

在中度弱网条件下，使用 15fps 和较低码率后，画面流畅性基本可接受，能够满足巡检人员观察设备整体状态的需求。在严重弱网条件下，系统需要进一步降低分辨率或等待网络恢复，此时视频细节识别能力下降，但能够减少连接频繁中断。



图 6-2 不同带宽下视频画质对比截图-good



图 6-3 不同带宽下视频画质对比截图-weak



图 6-4 不同带宽下视频画质对比截图-poor

表 6-5 带宽变化与降级效果测试表

测试阶段	网络等级	分辨率	帧率	码率	现象
阶段一	good	1280x720	25fps	2000kbps	画面清晰
阶段二	weak	960x540	20fps	1200kbps	清晰下降
阶段三	poor	640x360	15fps	600kbps	细节减少
阶段四	recovery	1280x720	25fps	2000kbps	网络恢复

6.3.4 连续波动网络下稳定性测试

连续波动网络测试主要模拟机器人移动过程中信号强弱反复变化的情况。测试时交替设置正常带宽和低带宽，并观察前端播放状态、后端设备状态和采集端日志。

测试结果表明，在短时波动场景下，WebRTC 播放可通过重连机制恢复；在波动持续时间较长时，前端可能出现短暂加载或错误提示，但当采集端继续推流且网络恢复后，播放可重新建立。在心跳超过阈值未恢复时，后端会将设备置为离线，前端刷新后显示离线状态。

该测试说明，系统在连续波动网络下具有一定容错能力，但仍需要进一步优化自动码率调整的实时性和精度。



图 6-5 连续波动网络下播放状态变化-加载



图 6-6 连续波动网络下播放状态变化-重连中



图 6-7 连续波动网络下播放状态变化-无法连接设备

6.4 性能观察

6.4.1 端到端时延观察

查重 51%
端到端时延是评价实时视频系统的重要指标。测试方法为在摄像头前显示带毫秒时间的计时器，浏览器端播放画面后拍摄或截图比较实际时间差。也可通过在现场做明显动作并观察浏览器显示延迟进行粗略测量。

在局域网环境下，系统使用 WebRTC 播放能够满足实时监控对低延迟的基本要求。测试过程中，通过前端性能监控面板的 RTT 指标获取网络层时延数据，结合主观观察得出端到端时延。**查重 41%**
测试结果显示，正常网络下平均时延约 0.7 秒，最大不超过 1.0 秒；轻度弱网下平均时延约 1.1 秒，最大约 1.5 秒；严重弱网下平均时延约 1.4 秒，最大约 2.0 秒。由于系统采用 MediaMTX 直接转发媒体流，FastAPI 后端不参与视频数据转发，因此业务接口负载对视频时延影响较小。

性能监控面板显示的 RTT 指标反映了 WebRTC 连接的网络往返时延，正常网络下 RTT 通常在 50-100ms 之间，轻度弱网下 RTT 上升到 100-200ms，严重弱网下 RTT 可能超过 200ms。该指标为弱网降级决策提供了量化依据。

表 6-6 端到端时延观察结果表

场景	视频参数	观察方式	时延范围	RTT 范围	结论
正常网络	1280x720、 25fps、2000kbps	计时器	平均 0.7s, 最大 1.0s	50	100ms
轻度弱网	960x540、 20fps、1200kbps	人工观察	平均 1.1s, 最大 1.5s	100	200ms
严重弱网	640x360、 15fps、600kbps	人工观察	平均 1.4s, 最大 2.0s	>200ms	可维持画面

```

(venv) yuxiang@yingraspberrypi:~/Desktop/video/camera-client $ python main.py
2026-05-23 18:15:37 | INFO | __main__:setup - 摄像头采集端启动 mode=WHIP
2026-05-23 18:15:37 | INFO | device_manager:register - 注册设备 name=Camera-001 type=robot location=2号机组 ip=192.1
68.1.9
2026-05-23 18:15:38 | INFO | device_manager:register - 设备注册成功 device=940d3c37-ebc6-4f0f-ad1a-7d55ad728103
2026-05-23 18:15:38 | INFO | device_manager:start_heartbeat - 心跳已启动, 间隔: 10秒
2026-05-23 18:15:38 | INFO | __main__:start_streamer - 启动推流 profile=good resolution=1280x720 fps=25 bitrate=2000
kbps
2026-05-23 18:15:38 | INFO | whip_streamer:start - 启动WHIP推流 url=http://192.168.1.5:8889/940d3c37-ebc6-4f0f-ad1a-
7d55ad728103/whip resolution=1280x720 fps=25 bitrate=2000kbps
[0:37:16.617594729] [3257] INFO Camera camera_manager.cpp:330 libcamera v0.5.2+99-bfd68f78
[0:37:16.62798059] [3265] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:50)
[0:37:16.630520813] [3265] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/share/libcamera/ipa/rpi/pisp/ov5647.j
son
[0:37:16.637404768] [3265] INFO Camera camera_manager.cpp:220 Adding camera '/base/axi/pcie@120000/rp1/i2c@88000/ov5647
@36' for pipeline handler rpi/pisp
[0:37:16.637427064] [3265] INFO RPI pisp.cpp:1179 Registered camera /base/axi/pcie@120000/rp1/i2c@88000/ov5647@36 to CF
E device /dev/media0 and ISP device /dev/media3 using PiSP variant BCM2712_C0
[0:37:16.637475045] [3265] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:50)
[0:37:16.638991605] [3265] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/share/libcamera/ipa/rpi/pisp/imx290.j
son
[0:37:16.640989198] [3265] INFO Camera camera_manager.cpp:220 Adding camera '/base/axi/pcie@120000/rp1/i2c@88000/imx290
@1a' for pipeline handler rpi/pisp
[0:37:16.641012920] [3265] INFO RPI pisp.cpp:1179 Registered camera /base/axi/pcie@120000/rp1/i2c@88000/imx290@1a to CF
E device /dev/media1 and ISP device /dev/media4 using PiSP variant BCM2712_C0
[0:37:16.643794691] [3257] INFO Camera camera.cpp:1215 configuring streams: (0) 1280x720-RGB888/Rec709/Rec709/None/Full
(1) 1920x1080-GBRG_PISP_COMP1/RAW
[0:37:16.643940967] [3265] INFO RPI pisp.cpp:1483 Sensor: /base/axi/pcie@120000/rp1/i2c@88000/ov5647@36 - Selected sens
or format: 1920x1080-SGBRG10_1X10/RAW - Selected CFE format: 1920x1080-PC1g/RAW
2026-05-23 18:15:38 | INFO | device_manager:heartbeat_loop - 心跳正常 device=940d3c37-ebc6-4f0f-ad1a-7d55ad728103 r
tt=52.66ms net=good
2026-05-23 18:15:39 | INFO | whip_streamer:start - WHIP推流已启动
2026-05-23 18:15:39 | WARNING | __main__:report_stream_status - [档位变化] init → good | 分辨率: 1280x720 | 帧率: 25
fps | 码率: 2000kbps | RTT: 52.66ms | 丢包率: 0.0% | 重连次数: 0 | 流状态: active
2026-05-23 18:15:39 | INFO | __main__:setup - 初始化完成
2026-05-23 18:15:39 | INFO | __main__:run - WHIP推流运行中...
2026-05-23 18:15:39 | INFO | whip_streamer:on_connectionstatechange - WHIP连接状态 state=connecting
2026-05-23 18:15:39 | INFO | __main__:report_stream_status - 采集状态 stream=connecting net=good rtt=52.66ms loss=0.
0% reconnects=0 profile=1280x720@25fps/2000kbps
2026-05-23 18:15:39 | INFO | whip_streamer:on_connectionstatechange - WHIP连接状态 state=connected
2026-05-23 18:15:39 | INFO | __main__:report_stream_status - 采集状态 stream=active net=good rtt=52.66ms loss=0.0% r
econnects=0 profile=1280x720@25fps/2000kbps
2026-05-23 18:17:19 | INFO | device_manager:heartbeat_loop - 心跳正常 device=940d3c37-ebc6-4f0f-ad1a-7d55ad728103 r
tt=38.15ms net=good
2026-05-23 18:18:37 | WARNING | whip_streamer:on_connectionstatechange - WHIP连接状态 state=closed
2026-05-23 18:18:37 | INFO | __main__:report_stream_status - 采集状态 stream=reconnecting net=good rtt=3105.75ms los
s=0.0% reconnects=0 profile=1280x720@25fps/2000kbps
2026-05-23 18:18:40 | INFO | __main__:report_stream_status - 采集状态 stream=reconnecting net=good rtt=3105.75ms los
s=0.0% reconnects=1 profile=1280x720@25fps/2000kbps
2026-05-23 18:18:41 | INFO | whip_streamer:stop - 停止WHIP推流...
2026-05-23 18:18:41 | INFO | whip_streamer:stop - WHIP推流已停止
2026-05-23 18:18:44 | WARNING | __main__:restart_streamer - [重建推流] profile=weak | 分辨率: 960x540 | 帧率: 20fps
| 码率: 1200kbps | 重连次数: 1
2026-05-23 18:18:44 | INFO | __main__:start_streamer - 启动推流 profile=weak resolution=960x540 fps=20 bitrate=1200k
bps
2026-05-23 18:18:44 | INFO | whip_streamer:start - 启动WHIP推流 url=http://192.168.1.5:8889/940d3c37-ebc6-4f0f-ad1a-

```

图 6-8 性能监控面板显示的实时指标截图

6.4.2 不同带宽下卡顿现象对比

卡顿现象对比主要观察在固定测试时长内视频播放是否出现停顿、加载或明显跳帧。测试分别在正常带宽、轻度弱网、中度弱网和严重弱网条件下进行。

测试过程中，通过前端性能监控面板观察丢包率变化。正常网络下丢包率通常低于 2%，轻度弱网下丢包率上升到 3-5%，严重弱网下丢包率可能超过 10%。丢包率是影响视频卡顿的重要因素，当丢包率超过 5%时，视频播放会出现明显的停顿或跳帧。

测试结果表明，在正常带宽下，系统播放较稳定，卡顿较少；在轻度弱网下，通过降低码率后仍能保持较好的连续性；在中度弱网下，降低帧率后可明显减少停顿；在严重弱网下，视频清晰度和帧率下降较明显，但低参数传输仍优于高参数传输。由此可见，弱网降级策略能够在一定程度上降低卡顿概率。

```
2026-05-23 18:15:39 | WARNING | __main__:report_stream_status - [档位变化] init → good | 分辨率: 1280x720 | 帧率: 25
fps | 码率: 2000kbps | RTT: 52.66ms | 丢包率: 0.0% | 重连次数: 0 | 流状态: active
2026-05-23 18:15:39 | INFO | __main__:setup - 初始化完成
2026-05-23 18:15:39 | INFO | __main__:run - WHIP推流运行中...
2026-05-23 18:15:39 | INFO | whip_streamer:on_connectionstatechange - WHIP连接状态 state=connecting
2026-05-23 18:15:39 | INFO | __main__:report_stream_status - 采集状态 stream=connecting net=good rtt=52.66ms loss=0.
0% reconnects=0 profile=1280x720@25fps/2000kbps
2026-05-23 18:15:39 | INFO | whip_streamer:on_connectionstatechange - WHIP连接状态 state=connected
2026-05-23 18:15:39 | INFO | __main__:report_stream_status - 采集状态 stream=active net=good rtt=52.66ms loss=0.0% r
econnects=0 profile=1280x720@25fps/2000kbps
2026-05-23 18:17:19 | INFO | device_manager:heartbeat_loop - 心跳正常 device=940d3c37-ebc6-4f0f-ad1a-7d55ad728103 r
tt=38.15ms net=good
2026-05-23 18:18:37 | WARNING | whip_streamer:on_connectionstatechange - WHIP连接状态 state=closed
2026-05-23 18:18:37 | INFO | __main__:report_stream_status - 采集状态 stream=reconnecting net=good rtt=3105.75ms los
s=0.0% reconnects=0 profile=1280x720@25fps/2000kbps
2026-05-23 18:18:40 | INFO | __main__:report_stream_status - 采集状态 stream=reconnecting net=good rtt=3105.75ms los
s=0.0% reconnects=1 profile=1280x720@25fps/2000kbps
2026-05-23 18:18:41 | INFO | whip_streamer:stop - 停止WHIP推流...
2026-05-23 18:18:41 | INFO | whip_streamer:stop - WHIP推流已停止
2026-05-23 18:18:44 | WARNING | __main__:restart_streamer - [重建推流] profile=weak | 分辨率: 960x540 | 帧率: 20fps
| 码率: 1200kbps | 重连次数: 1
2026-05-23 18:18:44 | INFO | __main__:start_streamer - 启动推流 profile=weak resolution=960x540 fps=20 bitrate=1200k
bps
2026-05-23 18:18:44 | INFO | whip_streamer:start - 启动WHIP推流 url=http://192.168.1.5:8889/940d3c37-ebc6-4f0f-ad1a-
```

图 6-9 采集端档位变化日志截图

6.4.3 CPU/内存占用观察

CPU 和内存占用观察分别在采集端、后端服务、MediaMTX 和前端浏览器侧进行。采集端资源消耗主要来自摄像头采集和视频编码；MediaMTX 资源消耗主要来自流接入和转发；前端浏览器资源消耗随分屏数量增加而上升。

测试结果显示，树莓派 5 采集端在 good 档位下 CPU 占用约 35-45%，在 poor 档
位下降至 20-30%；MediaMTX 单路转发 CPU 占用约 5-10%；FastAPI 后端 CPU 占用
低于 5%；浏览器前端单路播放 CPU 占用约 15-25%，4 分屏时上升至 40-50%。内存
占用方面，各组件均在合理范围内，采集端约 200-300MB，MediaMTX 约 100-
150MB，后端约 80-120MB。

表 6-7 CPU 与内存占用观察表

组件	主要压力来源	单路视频表现	说明
采集端	摄像头采集	中等	CPU 占用高
MediaMTX	流接入与转发	较低	单路转发压力较小
FastAPI 后端	API 请求和心跳处理	较低	不直接转发媒体数据
浏览器前端	WebRTC 解码	中等	多分屏时占用上升

第7章 总结

7.1 工作总结

本文围绕电厂巡检机器人实时视频回传需求，设计并实现了一套端到端无线视频传输系统。系统采用前后端分离架构，结合摄像头采集端、FastAPI 后端、MediaMTX 流媒体服务和 Vue3 前端客户端，完成了从设备注册、心跳维护、视频采集、媒体推流、浏览器拉流到设备管理的核心链路。

在系统设计方面，本文分析了巡检机器人视频传输的业务场景和功能需求，设计了总体架构、模块划分、核心流程、数据库结构和接口方案。查重 40%在系统实现方面，后端完成用户认证、设备管理、心跳检测和视频状态管理；摄像头端完成设备注册、心跳上报和 WHIP 推流；MediaMTX 完成流接入和 WebRTC 分发；前端完成登录、设备列表、多分屏预览和播放控制。在系统测试方面，本文从功能测试、弱网测试和基础性能观察三个方面验证系统可用性。

总体来看，系统基本达到了毕业设计预期目标，能够在局域网环境下实现 720P 视频接入与低延迟预览，并具备设备状态管理和弱网恢复能力。

7.2 核心成果与创新点

查重 86%

本文的核心成果主要包括以下几个方面。

(1) 构建了一套以实时视频为核心的端到端传输系统。系统覆盖摄像头采集、设备注册、心跳检测、流媒体转发和浏览器播放等环节，不是单一的视频播放演示，而是面向实际巡检业务的轻量化工程系统。

(2) 采用 WHIP/WHEP 简化 WebRTC 接入。摄像头端通过 WHIP 推流，浏览器端通过 WHEP 拉流，避免自行设计复杂信令服务器，同时保持 WebRTC 低延迟优势。

(3) 实现设备管理与视频流路径统一。系统使用 `device_id` 作为设备唯一标识，也作为 MediaMTX 视频流路径，使后端设备数据和前端播放地址能够自然关联，降低了系统维护复杂度。

(4) 设计了合适的弱网自适应方案。系统通过参数降级、低延迟编码、播放端重连和心跳超时检测等机制，在工程复杂度可控的前提下提高弱网环境下的视频可用性。

7.3 不足与后续展望

虽然系统完成了主要功能，但仍存在一些不足。

(1) 弱网自适应策略仍较基础。当前系统主要依赖预设参数降级、播放端重连和心跳检测，尚未充分利用 WebRTC 中的 RTT、丢包率、码率和抖动等指标。后续可引入更精细的网络质量评估算法，实现真正的实时动态码率调整。

(2) 权限管理还不够完善。查重 72%当前系统尚未实现细粒度权限控制。后续可根据管理员、操作员和访客等角色限制设备删除和系统配置等操作。

查重 41%(3) 多设备并发和长时间稳定性测试还需加强。现阶段主要在有限设备和局域网环境下测试，后续应在更多摄像头、多客户端观看和长时间运行条件下验证系统性能。

(4) 安全性可进一步增强。生产环境部署时，建议进一步启用 HTTPS 加密传输、配置 MediaMTX 流媒体鉴权、定期更新密钥和令牌、配置防火墙规则等安全增强措施。

(5) 系统功能范围仍较集中。当前系统主要围绕实时视频、设备管理和弱网恢复展开，尚未覆盖更复杂的机器人任务调度、告警联动等业务能力。后续可在保持核心链路稳定的基础上逐步扩展。

综上所述，本文完成的系统具有较好的工程完整性。未来可在网络自适应、安全认证、多设备部署和 AI 辅助识别方面继续优化，使其更接近实际电厂巡检机器人应用需求。

参考文献

- [1] 张辉, 杜瑞, 钟杭, 曹意宏, 王耀南. 电力设施多模态精细化机器人巡检关键技术及应用[J]. 自动化学报, 2025, 51(1): 20-42.
- [2] IETF. WebRTC: Real-Time Communication in Browsers: RFC 8825[S]. 2021.
- [3] FastAPI Documentation. FastAPI Framework User Guide[EB/OL].
- [4] SQLAlchemy Documentation. SQLAlchemy ORM Documentation[EB/OL].
- [5] MediaMTX Documentation. MediaMTX Official Documentation[EB/OL].
- [6] Vue.js Documentation. Vue 3 Official Guide[EB/OL].
- [7] Element Plus Documentation. Element Plus Component Library[EB/OL].
- [8] IETF. WebRTC-HTTP ingestion protocol (WHIP): Internet-Draft draft-ietf-wish-whip-16[S]. 2025.
- [9] IETF. WebRTC-HTTP Egress Protocol (WHEP): Internet-Draft draft-ietf-wish-whep-03[S]. 2024.
- [10] 刘云浩. 物联网导论[M]. 北京: 科学出版社, 2013.
- [11] IETF. Real Time Streaming Protocol Version 2.0: RFC 7826[S]. 2016.
- [12] W3C. WebRTC 1.0: Real-Time Communication Between Browsers[EB/OL].
- [13] FFmpeg Documentation. FFmpeg Documentation[EB/OL].
- [14] aiortc Documentation[EB/OL].
- [15] 谢希仁. 计算机网络 (第 8 版) [M]. 北京: 电子工业出版社, 2021.

附录

附录 A 系统部署说明

A.1 Windows 环境部署（后端、前端、MediaMTX）

1. 查看本机局域网 IP 地址

打开命令提示符，执行：ipconfig

记录 IPv4 地址，例如：192.168.1.100

2. 启动 MediaMTX 流媒体服务

下载 MediaMTX Windows 版本，解压后执行：

```
./mediamtx.exe
```

服务启动后监听 8554（RTSP）、8889（WebRTC）、8888（HLS）端口。

3. 启动后端服务

进入 server 目录：

```
cd server
```

激活虚拟环境（PowerShell）：

```
venv\Scripts\Activate.ps1
```

安装依赖：

```
pip install -r requirements.txt
```

初始化数据库：

```
python -c "from models import init_db; import asyncio; asyncio.run(init_db())"
```

启动后端服务：

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

后端服务启动后监听 8000 端口。

4. 启动前端服务

进入 web-client 目录：

```
cd web-client
```

安装依赖：

```
pnpm install
```

启动开发服务器：

```
pnpm dev
```

前端服务启动后监听 5173 端口，浏览器访问 <http://localhost:5173>

A.2 Linux 环境部署（树莓派采集端）

1. 检查摄像头设备

列举连接的摄像头:

```
libcamera-hello --list-cameras
```

或检查 USB 摄像头:

```
ls -l /dev/video*
```

测试摄像头:

```
rpicam-hello
```

或:

```
libcamera-still -o test.jpg --width 640 --height 480
```

2. 配置采集端

进入采集端目录:

```
cd Desktop/video/camera-client
```

编辑配置文件:

```
nano .env
```

查重 56%

修改以下配置为后端服务器 IP 地址:

```
SERVER_URL=http://192.168.1.100:8000
```

保存 (Ctrl+O) 并退出 (Ctrl+X)

3. 启动采集端

激活虚拟环境:

```
source venv/bin/activate
```

安装依赖:

```
pip install -r requirements.txt
```

启动采集端:

```
python main.py
```

采集端启动后自动向后端注册设备并开始推流。

A.3 系统访问与使用

1. 浏览器访问前端地址: <http://192.168.1.100:5173>

2. 使用默认管理员账号登录:

用户名: admin

密码: admin123

致 谢

大学阶段的学习生活即将结束，回顾本次毕业设计从选题、需求分析、系统设计、代码实现到论文撰写的全过程，我对软件工程实践、实时视频传输技术和工业巡检应用场景都有了更加系统的认识。查重 71%在此，向所有给予我帮助和支持的老师、同学和家人表示诚挚感谢。查重 55%

首先，感谢指导教师在毕业设计过程中给予的耐心指导。从课题方向确定、技术方案选择到论文结构调整，老师都提出了许多宝贵意见，使我能够逐步明确系统目标，完善实现细节，并以更加规范的方式完成论文写作。查重 63%

其次，感谢同学们在项目调试和测试过程中给予的帮助。在系统开发过程中，我遇到了依赖安装、流媒体服务启动、摄像头权限、WebRTC 播放、弱网测试等问题，同学们的交流和建议帮助我更快定位问题，也让我意识到工程实践中沟通与协作的重要性。查重 55%

再次，感谢家人在学习和生活上的支持。正是他们的理解和鼓励，使我能够保持耐心完成毕业设计的各项工作。

最后，感谢开源社区提供的 FastAPI、Vue3、MediaMTX、FFmpeg、ElementPlus 等优秀工具和文档。这些开源项目为本系统的实现提供了坚实基础，也让我在实践中感受到现代软件生态的开放与高效。毕业设计虽然告一段落，但学习和探索仍会继续。未来我将持续提升工程能力和问题分析能力，把本次毕业设计中积累的经验应用到更多实际项目中。